

Artificial Intelligence

Piya Limcharoen

Version 1.0

Contents

Chapter 6: Advanced Deep Learning Architectures	5
1.1 Chapter Overview	5
1.2 Autoencoders (Data Compression)	7
1.2.1 The Basic Idea	7
1.2.2 The Three Main Parts	9
1.2.3 Learning to Reconstruct	10
1.2.4 How Autoencoders Learn: Unsupervised Reconstruction	15
1.2.5 Applications of Autoencoders	18
1.2.6 Python Implementation: TensorFlow/Keras, PCA Baseline & Visualization	33
1.3 Generative Adversarial Networks (Data Generation)	45
1.3.1 The Basic Idea	45
1.3.2 Why We Need GANs	50
1.3.3 How GANs Learn: Adversarial Optimization	51

1.3.4	PyTorch Implementation: DCGAN on Aligned Human Faces	55
1.3.5	Applications of GANs	69
1.4	Recurrent Neural Networks (Sequential Learning)	74
1.4.1	Architectural Design: Learning from Sequential Data	74
1.4.2	Why Sequential Data Is Different	74
1.4.3	The Problem with Basic RNNs	75
1.4.4	How LSTMs Learn	76
1.4.5	Applications of RNNs and LSTMs	80
1.5	Architectural Comparison: Autoencoders vs. GANs vs. LSTMs	89
1.6	Practice Questions & Solutions	90
1.7	Chapter Summary	94

Copyright Notice

Copyright © 2026 Piya Limcharoen. All rights reserved.

This lecture note was authored by Piya Limcharoen. Artificial Intelligence (AI) was utilized as a tool for language enhancement and grammatical correction; however, all intellectual property, original concepts, and the structure of this work belong exclusively to the author.

Terms of Use

This document is provided for educational purposes only and is intended for the exclusive use of students currently enrolled in this course.

The reproduction, distribution, public display, or transmission of this material, in whole or in part, in any form or by any means—electronic, mechanical, photocopying, recording, or otherwise—is strictly prohibited without the prior written permission of the author. Students may not share, upload, or distribute these notes outside of the official class environment.

Warning

Any unauthorized use of this material will be considered academic misconduct and will be subject to university disciplinary procedures. Furthermore, such an act may constitute an infringement of copyright law and is subject to legal action.

Chapter 6:

Advanced Deep Learning
Architectures

Chapter 6: Advanced Deep Learning Architectures

1.1 Chapter Overview

In previous chapters, we studied standard feedforward neural networks: Multi-Layer Perceptrons (MLPs) and Convolutional Neural Networks (CNNs). In those architectures, fixed-size inputs (such as tabular feature vectors or image grids) are mapped directly to predefined target labels through supervised learning.

However, real-world artificial intelligence requires models that can discover representations without labels, generate new synthetic content, or process sequences that unfold over time. In this chapter, we explore three foundational deep learning architectures designed for these challenges:

1. **Autoencoders (AEs):** Neural networks that learn compact representations by compressing and reconstructing unlabelled data, enabling applications such as dimensionality reduction, image denoising, anomaly detection, and vector embeddings for semantic search.
2. **Generative Adversarial Networks (GANs):** A two-network framework where a Generator and a Discriminator compete in an adversarial game to synthesize realistic new samples from noise.
3. **LSTMs and Recurrent Networks:** Sequential models that use gated memory mechanisms to maintain context across time, forming the foundation for Sequence-to-Sequence translation, language modeling, time-series forecasting, and modern Large Language Models (LLMs).

Learning Objectives

By the end of this chapter, you will be able to:

1. **Autoencoder Fundamentals:** Explain the structure of an Autoencoder (Encoder, Bottleneck latent space, Decoder) and how unsupervised reconstruction loss guides representation learning.
2. **Autoencoder Applications:** Describe how Autoencoders are applied to image denoising, industrial anomaly detection, and dense vector embeddings for Retrieval-Augmented Generation (RAG).
3. **GAN Adversarial Framework:** Explain the roles of the Generator and Discriminator, how adversarial optimization trains both networks, and why GANs generate sharper samples than standard Autoencoders.
4. **GAN Applications & Challenges:** Identify common GAN applications (image generation, synthetic data, style transfer) and understand challenges such as mode collapse and training instability.
5. **Sequential Data & RNN Limitations:** Explain why sequential data requires specialized models and how the vanishing gradient problem limits basic vanilla RNNs.
6. **LSTM Memory & Gating:** Explain how the Cell State and three gating mechanisms (Forget Gate, Input Gate, Output Gate) enable LSTMs to remember long-term dependencies.
7. **Sequence Applications & LLMs:** Describe how recurrent models power Sequence-to-Sequence translation and next-token language modeling, and explain their conceptual connection to modern Transformer-based LLMs.

1.2 Autoencoders (Data Compression)

1.2.1 The Basic Idea

An **Autoencoder (AE)** (โครงข่ายเข้ารหัสอัตโนมัติ) is a neural network that learns to represent data in a more compact form and then reconstruct the original data.

The basic process is:

$$\mathbf{x} \longrightarrow \mathbf{z} \longrightarrow \hat{\mathbf{x}}$$

where $\mathbf{x}$ is the original input, $\mathbf{z}$ is a lower-dimensional latent representation, and $\hat{\mathbf{x}}$ is the reconstructed input.

For example, if the input has $d = 784$ dimensions, the Autoencoder may compress it into a latent representation with only $k = 32$ dimensions:

$$\mathbf{x} \in \mathbb{R}^{784} \longrightarrow \mathbf{z} \in \mathbb{R}^{32} \longrightarrow \hat{\mathbf{x}} \in \mathbb{R}^{784}.$$

Thus, the latent representation has fewer dimensions than the original input:

$$k < d.$$

The model must therefore learn which information is important enough to preserve in these fewer dimensions so that the input can later be reconstructed.

What is an Autoencoder?

An autoencoder is a neural network that learns to compress input data into a compact representation and then reconstruct it back.

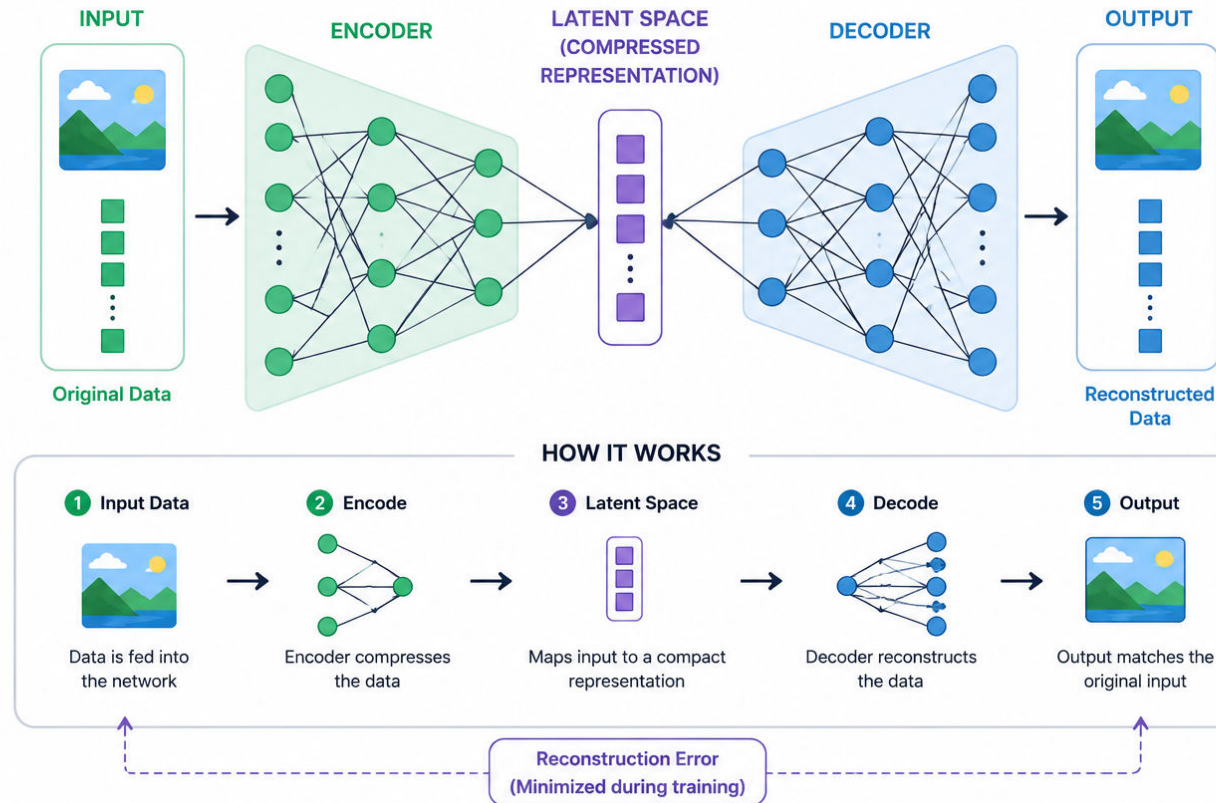


Figure 1: The Architecture and Working Principle of an Autoencoder: Compression of input data through an encoder network into a compact latent space representation, followed by reconstruction back to the original format via a decoder network, optimized by minimizing reconstruction error. (Source: <https://www.upgrad.com/blog/what-is-an-autoencoder/>)

1.2.2 The Three Main Parts

An Autoencoder consists of three main parts:

1. **Encoder:**

Converts the input $\mathbf{x} \in \mathbb{R}^d$ into a lower-dimensional representation $\mathbf{z} \in \mathbb{R}^k$, where $k < d$:

$$\mathbf{z} = f_{\boldsymbol{\theta}}(\mathbf{x}) \quad (1)$$

Here, $f_{\boldsymbol{\theta}}$ represents the encoder function with learnable parameters $\boldsymbol{\theta}$. It learns how to transform the input into a compact representation.

2. **Latent Space (Bottleneck):**

Stores the compact representation $\mathbf{z}$. Since $\mathbf{z}$ has fewer dimensions than the input, the model must learn to preserve the most important information needed for reconstruction.

3. **Decoder:**

Converts the latent representation $\mathbf{z}$ back into a reconstruction $\hat{\mathbf{x}}$:

$$\hat{\mathbf{x}} = g_{\boldsymbol{\phi}}(\mathbf{z}) \quad (2)$$

Here, $g_{\boldsymbol{\phi}}$ represents the decoder function with learnable parameters $\boldsymbol{\phi}$. It learns how to transform the latent representation back into the original data space.

1.2.3 Learning to Reconstruct

The Autoencoder is trained to make the reconstructed output $\hat{\mathbf{x}}$ as similar as possible to the original input $\mathbf{x}$. A common reconstruction loss is the mean squared error:

$$\mathcal{L} = \frac{1}{d} \sum_{i=1}^d (x_i - \hat{x}_i)^2 \quad (3)$$

By minimizing this loss, the network learns a latent representation that preserves the information needed to reconstruct the input.

Key Idea

An Autoencoder learns to **compress and reconstruct** data:

Input $\rightarrow$ Encoder $\rightarrow$ Latent Representation $\rightarrow$ Decoder $\rightarrow$ Reconstruction

The learned latent representation can also be useful for tasks such as **data compression, noise removal, and feature extraction**.

The Hourglass Architecture of an Autoencoder: Non-linear Feature Compression

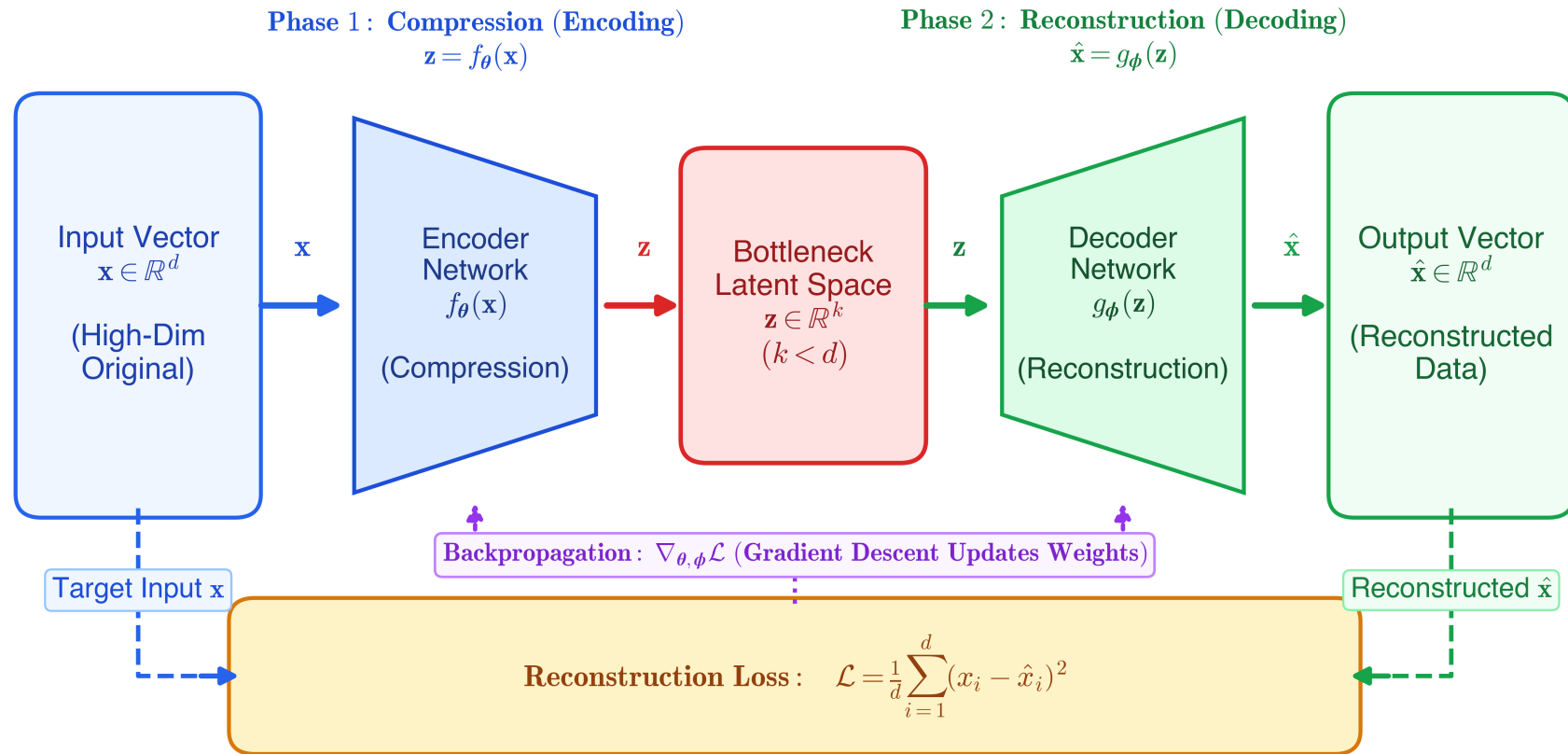


Figure 2: The Hourglass Architecture of an Autoencoder: non-linear feature compression of input $\mathbf{x} \in \mathbb{R}^d$ into bottleneck $\mathbf{z} \in \mathbb{R}^k$ ($k < d$) and decompression into reconstructed output $\hat{\mathbf{x}} \in \mathbb{R}^d$ trained via reconstruction loss $\mathcal{L} = \frac{1}{d} \sum_{i=1}^d (x_i - \hat{x}_i)^2$.

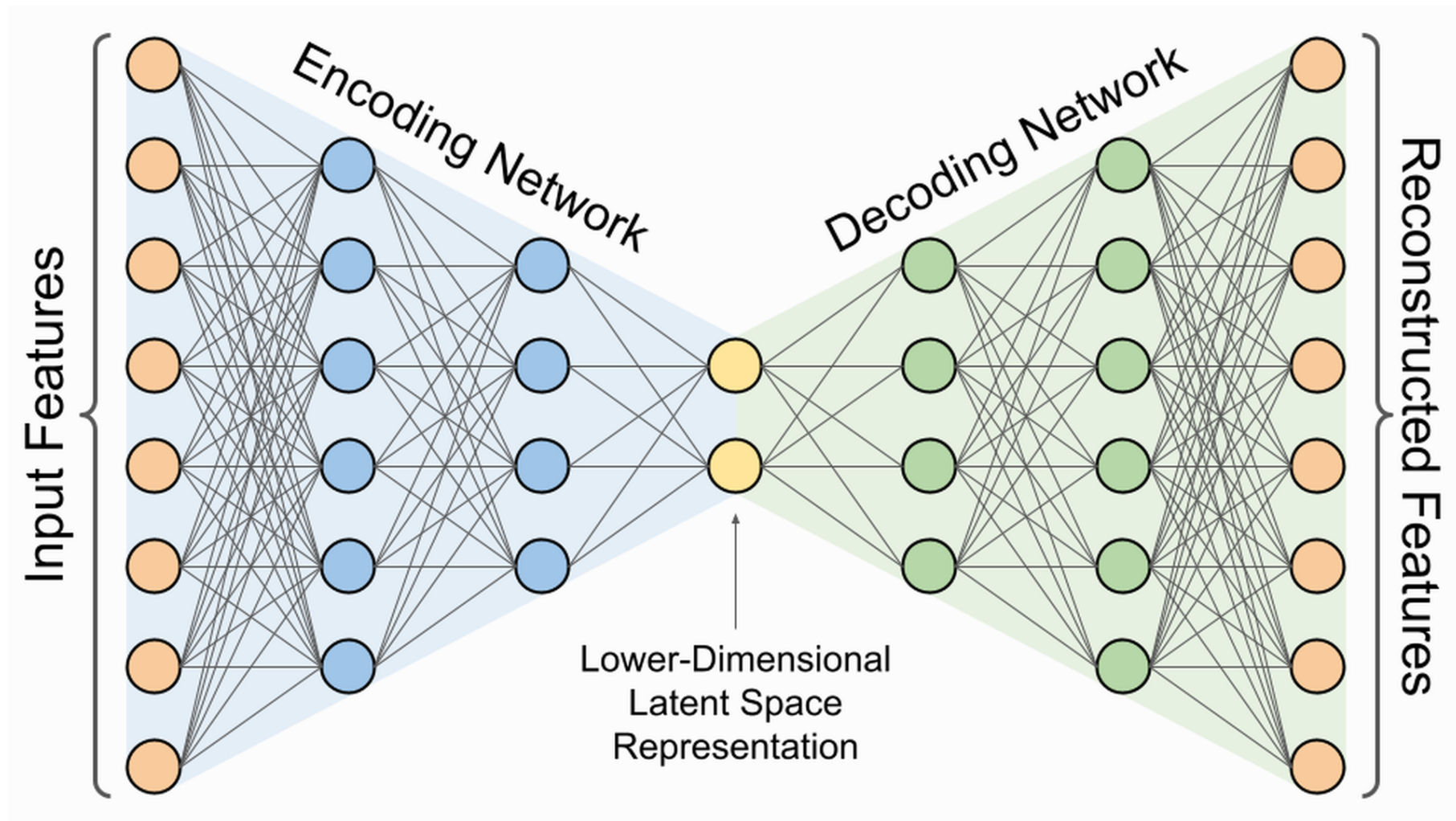


Figure 3: Layer-by-Layer Autoencoder Topology: The Encoding Network progressively compresses input features through multi-layer dense transformations into a lower-dimensional latent space representation (bottleneck), which the Decoding Network expands to produce the reconstructed output. (Source: <https://www.linkedin.com/pulse/autoencoders-pca-bridging-gap-machine-learning-sehrish-iqbal-93gff>)

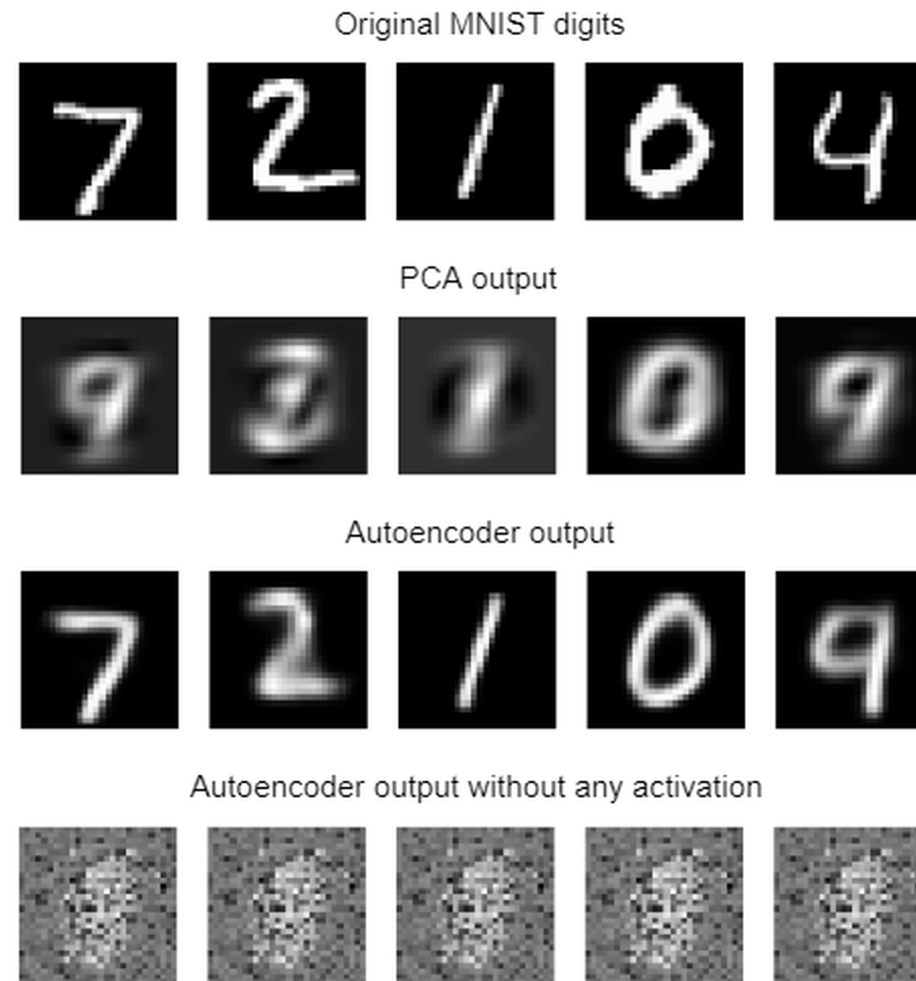


Figure 4: Reconstruction Performance on MNIST Digits: (Row 1) **Original MNIST digits**; (Row 2) **PCA Output**: Linear projection yields blurry reconstructions constrained to a flat linear subspace; (Row 3) **Non-linear Autoencoder Output**: Non-linear activation functions (e.g., ReLU, Sigmoid) enable learning curved non-linear manifolds, recovering sharp digit contours and distinct strokes; (Row 4) **Autoencoder Output without Activation Functions**: When non-linear activations are omitted, stacked dense layers mathematically collapse into a single composite linear transformation ($\hat{\mathbf{x}} = \mathbf{W}_{\text{dec}} \mathbf{W}_{\text{enc}} \mathbf{x} + \mathbf{b}$). (Source: <https://towardsdatascience.com/how-autoencoders-outperform-pca-in-dimensionality-reduction-1ae44c68b42f/>)

Linear vs nonlinear dimensionality reduction

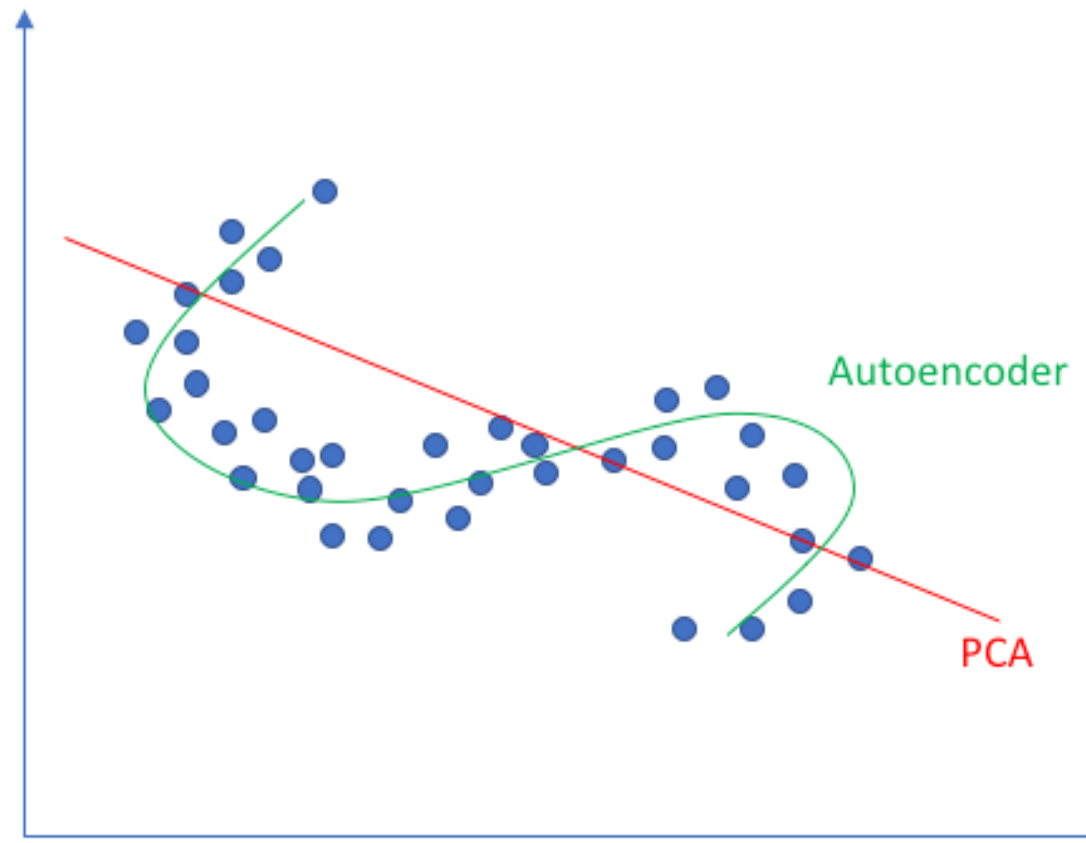


Figure 5: Linear vs. Non-linear Dimensionality Reduction: Principal Component Analysis (PCA) is constrained to finding a linear hyper-plane (straight line) across the data space, whereas an Autoencoder with non-linear activation functions learns a complex, curved low-dimensional manifold capturing non-linear data distributions. (Source: https://www.researchgate.net/publication/340049776_Fully_Convolutional_Variational_Autoencoder_For_Feature_Extraction_Of_Fire_Detection_System?_tp=eyJjb250ZXh0Ijp7ImZpcnN0UGFnZSI6I19kaXJlY3QiLCJwYXd1IjoiX2RpcmVjdCJ9fQ)

1.2.4 How Autoencoders Learn: Unsupervised Reconstruction

1.2.4.1 The Learning Mechanism

Autoencoders were introduced in late 1980s¹, they learn in an **unsupervised** manner, where the input $\mathbf{x}$ is also used as the target. The network compares the reconstructed output $\hat{\mathbf{x}}$ with the original input $\mathbf{x}$ using a **reconstruction loss**:

$$\mathcal{L}_{\text{recon}} = \frac{1}{d} \|\mathbf{x} - \hat{\mathbf{x}}\|^2 = \frac{1}{d} \|\mathbf{x} - g_{\phi}(f_{\theta}(\mathbf{x}))\|^2 \quad (4)$$

The goal is to minimize this loss so that the reconstructed output becomes as similar as possible to the original input.

Variables:

- $\mathbf{x} \in \mathbb{R}^d$: Original input vector.
- $f_{\theta}(\cdot)$: Encoder function with learnable parameters θ .
- $\mathbf{z} \in \mathbb{R}^k$: Lower-dimensional latent representation, where $k < d$.
- $g_{\phi}(\cdot)$: Decoder function with learnable parameters ϕ .
- $\hat{\mathbf{x}} \in \mathbb{R}^d$: Reconstructed output.

During training, **backpropagation** calculates the gradients of the reconstruction loss with respect to the parameters of both the encoder and decoder. Gradient descent then updates θ and ϕ to reduce the reconstruction loss.

¹The basic idea of an autoencoder for learning compressed representations is much older than modern deep learning.

Why Does the Bottleneck Size Matter?

- **Undercomplete Bottleneck** ($k < d$):

A smaller latent space forces the model to represent the input using fewer dimensions. This encourages the model to learn important information rather than simply copying the input.

- **Overcomplete Bottleneck** ($k \geq d$):

A larger latent space gives the model enough capacity to potentially learn an identity mapping, where $\hat{\mathbf{x}} \approx \mathbf{x}$ **without learning a useful compact representation.**

Autoencoder (AE) vs. Variational Autoencoder (VAE)

- **AE:** Learns a compact latent representation $\mathbf{z}$ for reconstructing the input.
- **VAE:** Learns a probability distribution of $\mathbf{z}$, allowing the model to generate new data.

$$\text{AE: } \mathbf{x} \rightarrow \mathbf{z} \rightarrow \hat{\mathbf{x}} \quad \text{VAE: } \mathbf{x} \rightarrow \text{Distribution}(\mathbf{z}) \rightarrow \hat{\mathbf{x}}$$

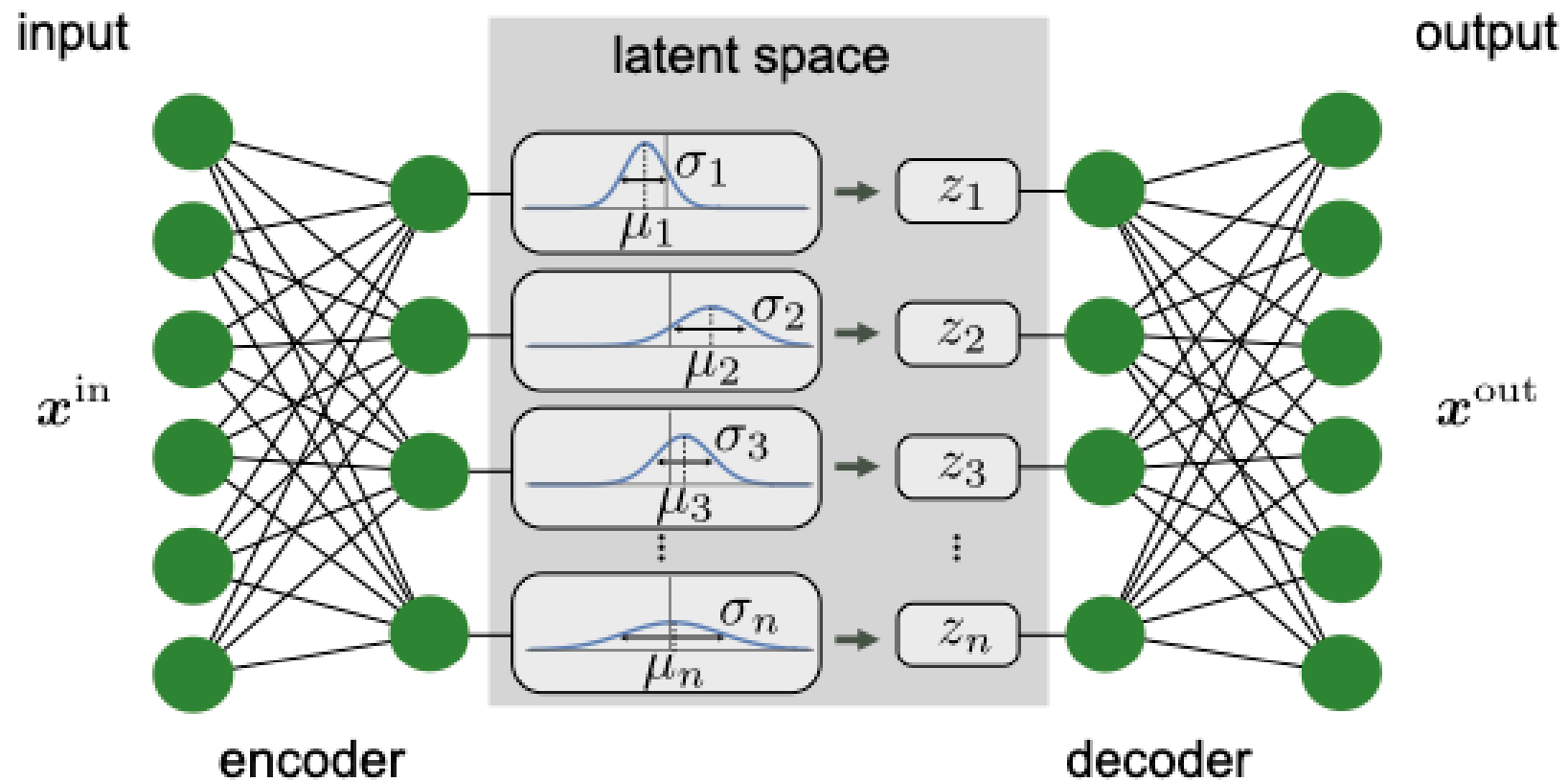


Figure 6: Variational Autoencoder (VAE) Architecture: The encoder outputs the parameters of probability distributions (mean μ_i and standard deviation σ_i) in the latent space, from which latent variables z_i are sampled and reconstructed by the decoder. (Source: https://ml-lectures.org/docs/unsupervised_learning/ml_unsupervised-3.html)

1.2.5 Applications of Autoencoders

1.2.5.1 Image Denoising (Denoising Autoencoders / DAE)

Autoencoders can be used to remove noise from images. For example, medical scans and low-light images may contain unwanted noise.

- **Setup:** Feed a noisy image $\tilde{\mathbf{x}}$ into the encoder, while using the original clean image $\mathbf{x}$ as the target:

$$\tilde{\mathbf{x}} = \mathbf{x} + \text{noise} \quad \longrightarrow \quad \text{Autoencoder} \quad \longrightarrow \quad \hat{\mathbf{x}} \approx \mathbf{x}.$$

- **Why it Works:** The model learns to preserve the important visual structure while removing noise that does not consistently appear in the training data.

Limitation: Why Autoencoders Produce Blurry Images

Because standard Autoencoders optimize Mean Squared Error (MSE), the model tends to output the pixel-wise average of all plausible reconstructions. For high-frequency details (such as hair, fabric textures, or foliage), this mathematical averaging results in smooth, blurry outputs. This limitation motivated the development of Generative Adversarial Networks (GAN).

Image Denoising Pipeline: Denoising Autoencoder (DAE)

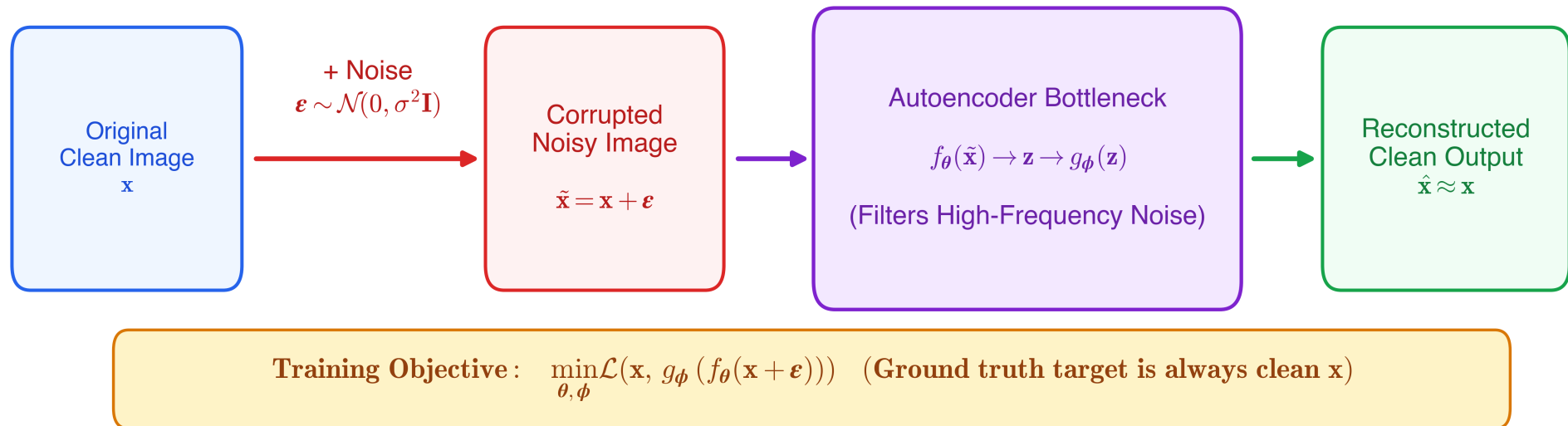


Figure 7: Image Denoising Pipeline: Denoising Autoencoder (DAE) training setup where artificial stochastic noise (such as zero-mean Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$ or masking noise) is added to the clean input $\mathbf{x}$ to produce a corrupted noisy image $\tilde{\mathbf{x}} = \mathbf{x} + \epsilon$. The autoencoder compresses $\tilde{\mathbf{x}}$ through the bottleneck latent representation $\mathbf{z}$, forcing the network to filter out unstructured high-frequency noise and reconstruct the clean uncorrupted output $\hat{\mathbf{x}} \approx \mathbf{x}$, optimized by minimizing reconstruction loss $\mathcal{L}(\mathbf{x}, \hat{\mathbf{x}})$ against the original clean target $\mathbf{x}$.

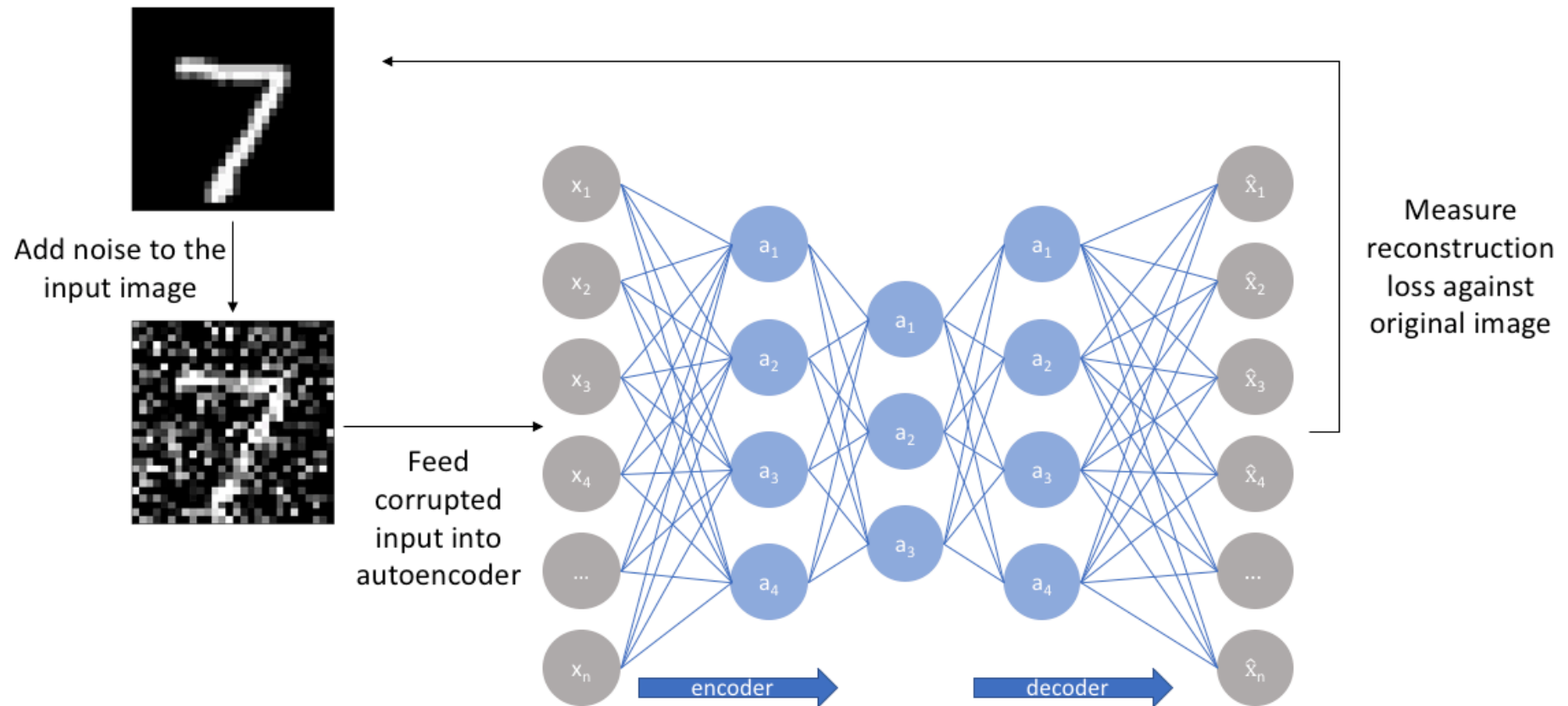


Figure 8: Denoising Autoencoder Training Pipeline: Noise is artificially injected into the input digit image, and the autoencoder is trained to reconstruct the original, uncorrupted clean image by minimizing reconstruction loss. (Source: <https://www.jeremyjordan.me/autoencoders/>)

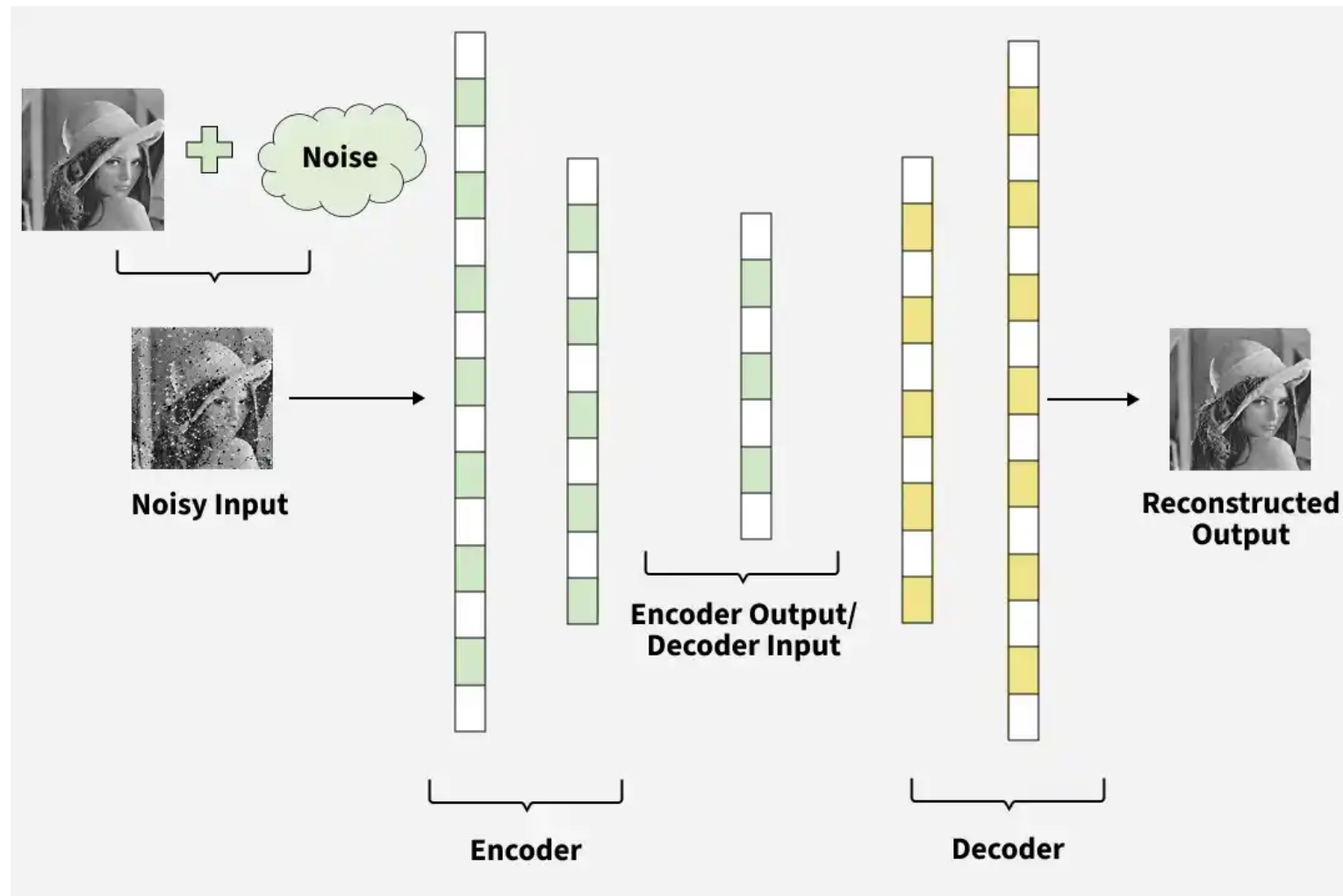


Figure 9: Denoising Autoencoder (DAE) Architecture: Intentionally injecting noise into the input forces the multi-layer Encoder to extract stable structural features into the latent bottleneck, enabling the Decoder to reconstruct a clean, noise-free output. (Source: <https://www.geeksforgeeks.org/machine-learning/denoising-autoencoders-in-machine-learning/>)

1.2.5.2 Industrial Anomaly Detection and Quality Control

Autoencoders can detect abnormal samples in applications such as manufacturing inspection and fraud detection.

1. **Train on Normal Data:** Train the Autoencoder using only normal, defect-free samples.
2. **Evaluate a New Sample:** Pass a new sample $\mathbf{x}_{\text{test}}$ through the trained model and calculate its reconstruction error:

$$\text{Anomaly Score} = \|\mathbf{x}_{\text{test}} - \hat{\mathbf{x}}_{\text{test}}\|^2 \quad (5)$$

3. **Make a Decision:** If the anomaly score exceeds a threshold τ , classify the sample as defective:

$$\text{Anomaly Score} > \tau \implies \text{Defect.}$$

Industrial Anomaly Detection via Reconstruction Thresholding

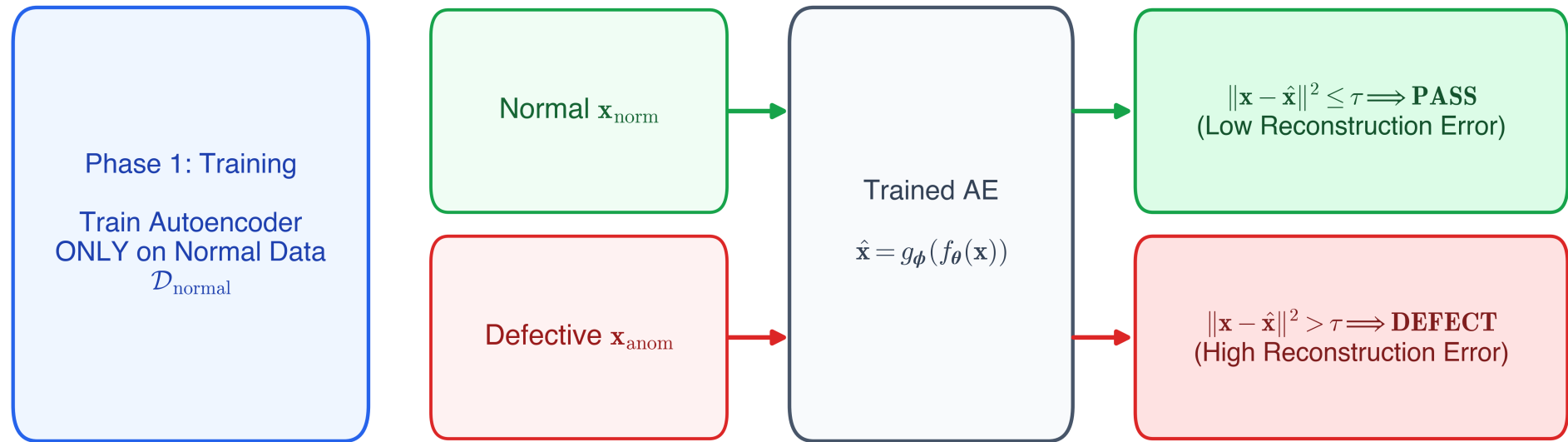


Figure 10: Industrial Anomaly Detection via Reconstruction Thresholding: The Autoencoder is trained strictly on normal data ($\mathcal{D}_{\text{normal}}$); query samples with high reconstruction error ($\|\mathbf{x} - \hat{\mathbf{x}}\|^2 > \tau$) are flagged as defects.

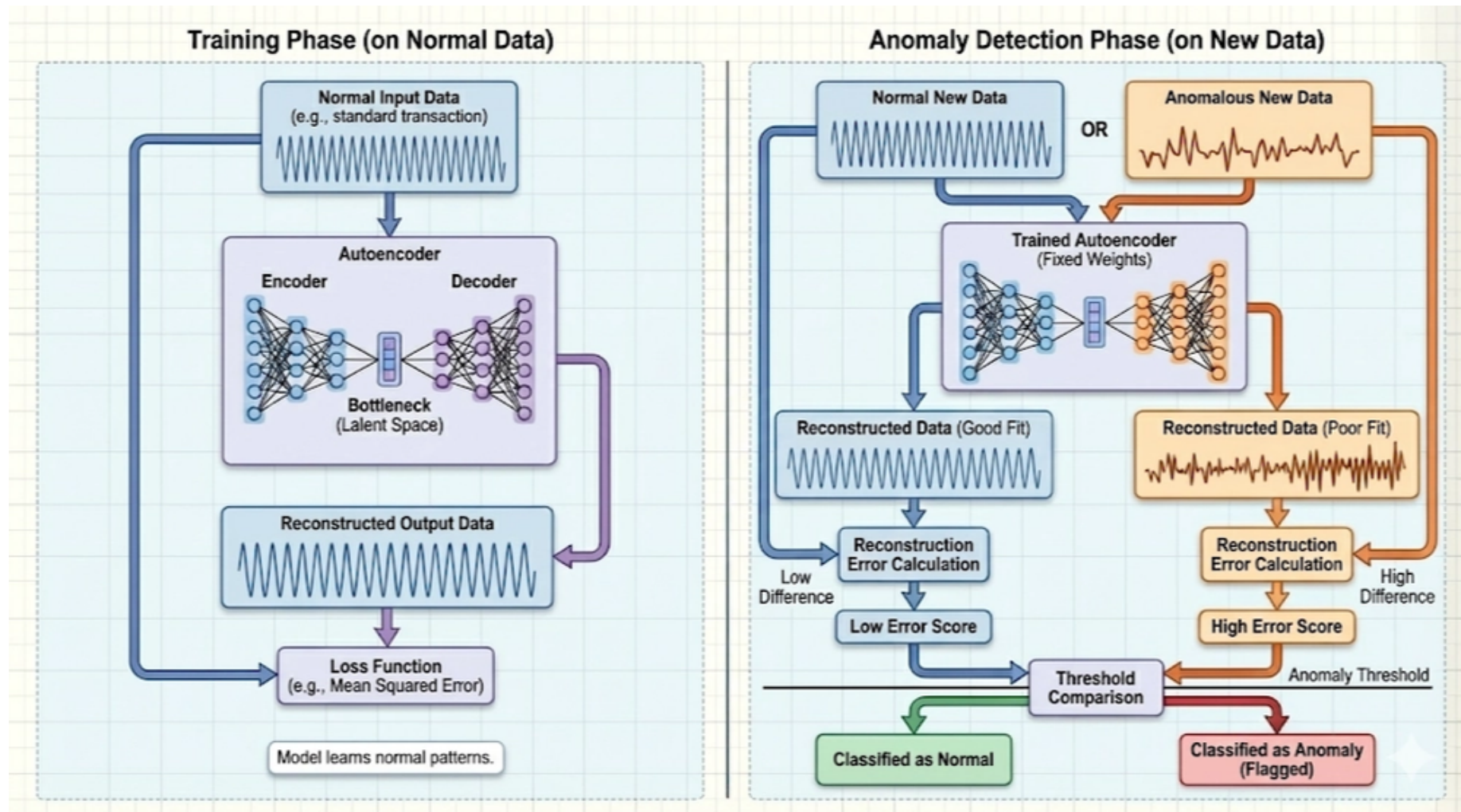


Figure 11: End-to-End Workflow for Anomaly Detection: Training Phase on normal data to minimize reconstruction loss (left), and Anomaly Detection Phase comparing query sample reconstruction error against a threshold τ to flag anomalies (right). (Source: <https://www.edgeimpulse.com/blog/a-convolutional-autoencoder-for-time-series-data-anomaly-detection/>)

1.2.5.3 Dense Text Embeddings for Vector Databases & RAG in LLMs

In modern Natural Language Processing and Large Language Models (LLMs), text embeddings are used to represent text as dense vectors that capture semantic meaning:

- **Semantic Representation:**

Text encoders convert sentences or documents into dense vectors (e.g., 768- or 1536-dimensional embeddings), similar to how an autoencoder represents data using a latent code $\mathbf{z}$.

- **Retrieval-Augmented Generation (RAG):**

RAG systems store document embeddings in vector databases such as Pinecone, Milvus, or Chroma. When a user asks a question, the system uses vector similarity to retrieve relevant documents and provides them to the LLM as context.

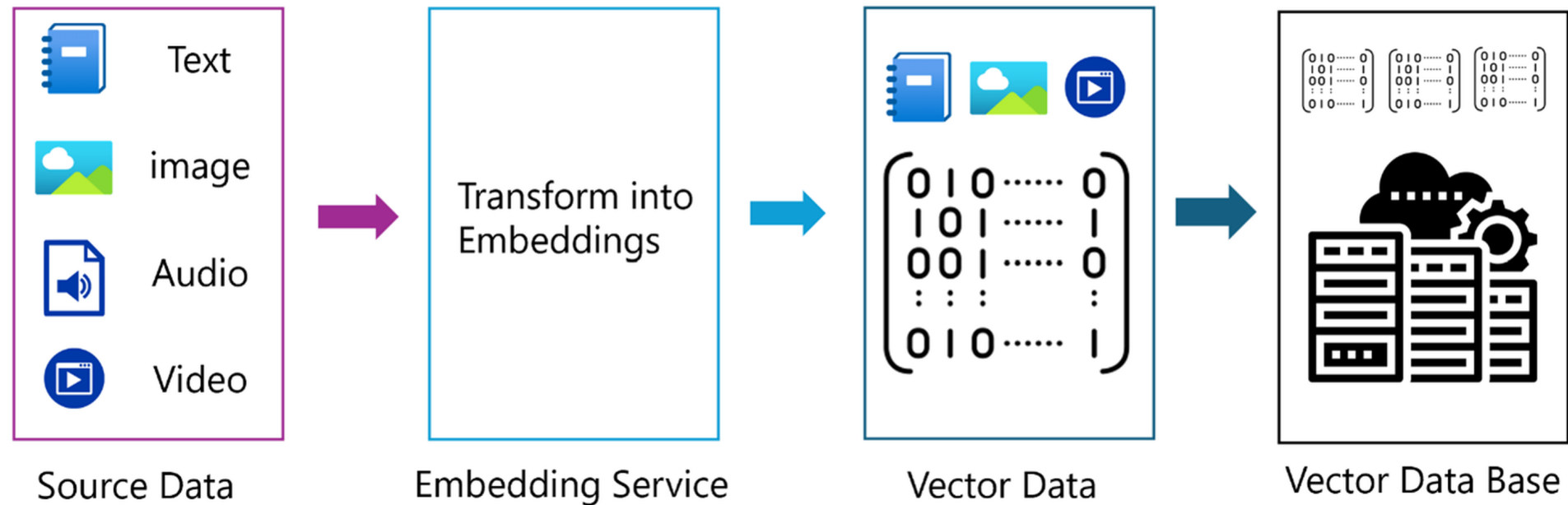


Figure 12: Vector Database and Embedding Pipeline for RAG: Multi-modal source data (text, images, audio, video) is transformed via deep neural embedding encoders into high-dimensional latent vectors and indexed within vector databases for ultra-fast semantic similarity search and LLM grounding. (Source:

<https://learn.microsoft.com/en-us/data-engineering/playbook/solutions/vector-database/>)

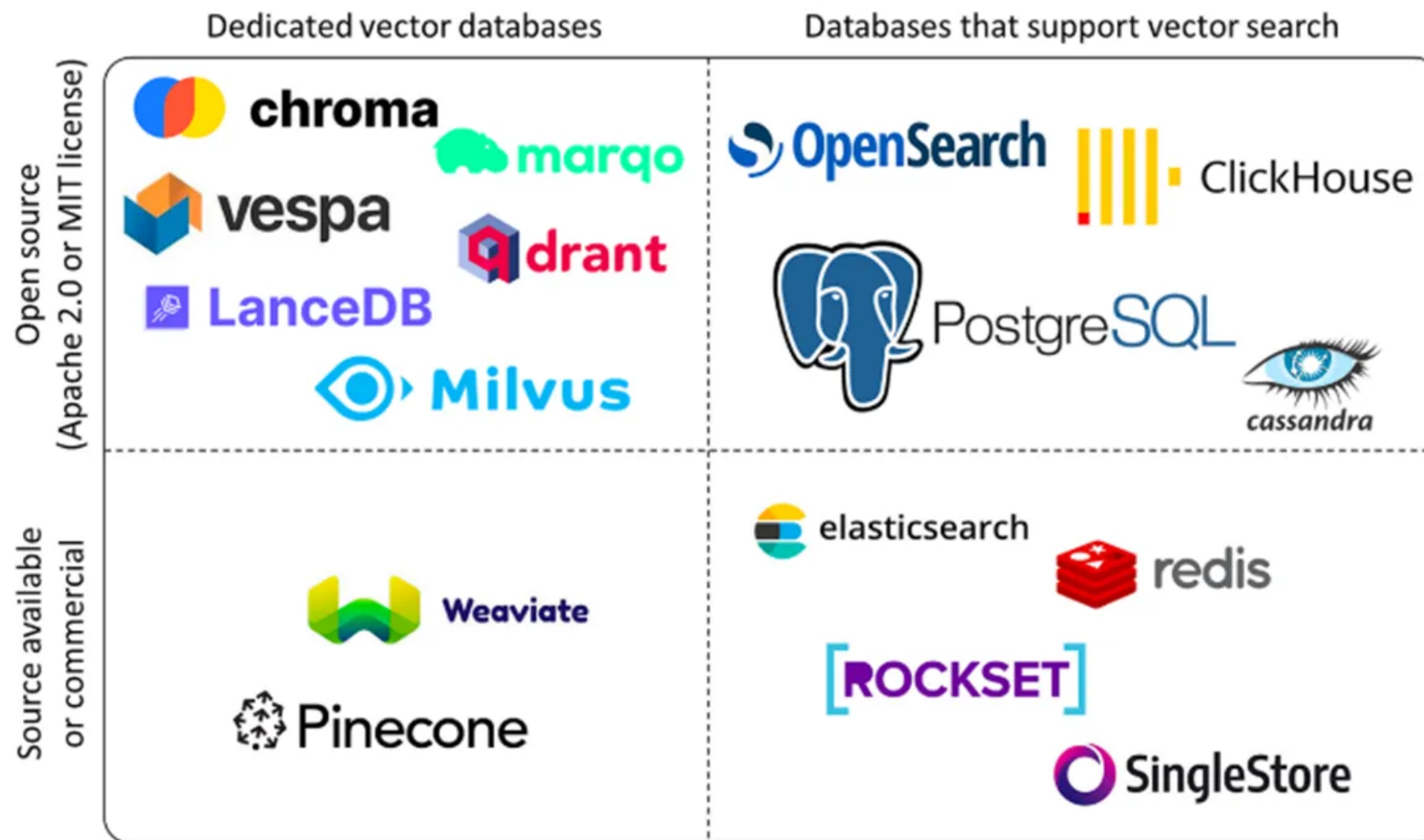


Figure 13: Vector Database Landscape and Ecosystem: Classification across Dedicated vector databases vs. Databases supporting vector search, and Open-source (Chroma, Milvus, Qdrant, Vespa) vs. Commercial / Source-available solutions (Pinecone, Weaviate, Redis, pgvector).
 (Source: <https://mehmetozkaya.medium.com/exploring-vector-databases-pinecone-chroma-weaviate-qdrant-milvus-pgvector-and-redis-f0618fe9e92d>)

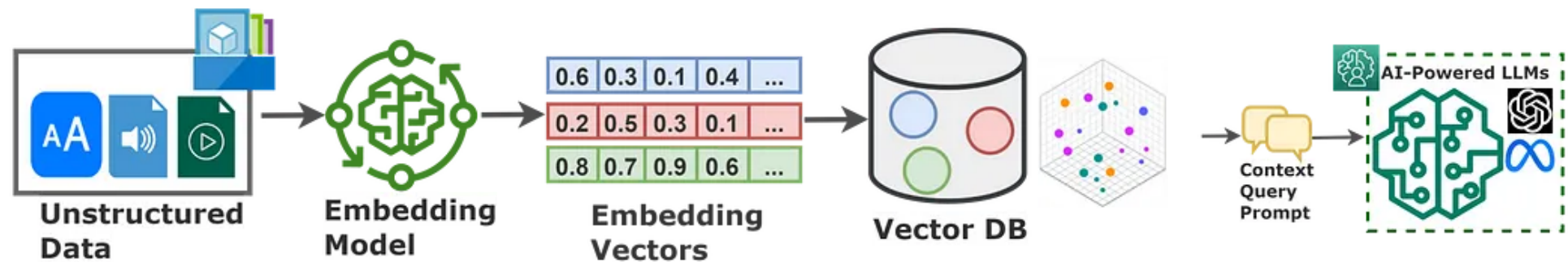


Figure 14: End-to-End Vector Search and RAG Architecture: Ingesting unstructured multi-modal data through an Embedding Model into dense vector representations, indexing within a Vector Database, and performing nearest-neighbor semantic search to augment LLM prompts with relevant context. (Source: <https://mehmetozkaya.medium.com/exploring-vector-databases-pinecone-chroma-weaviate-qdrant-milvus-pgvector-and-redis-f0618fe9e92d>)

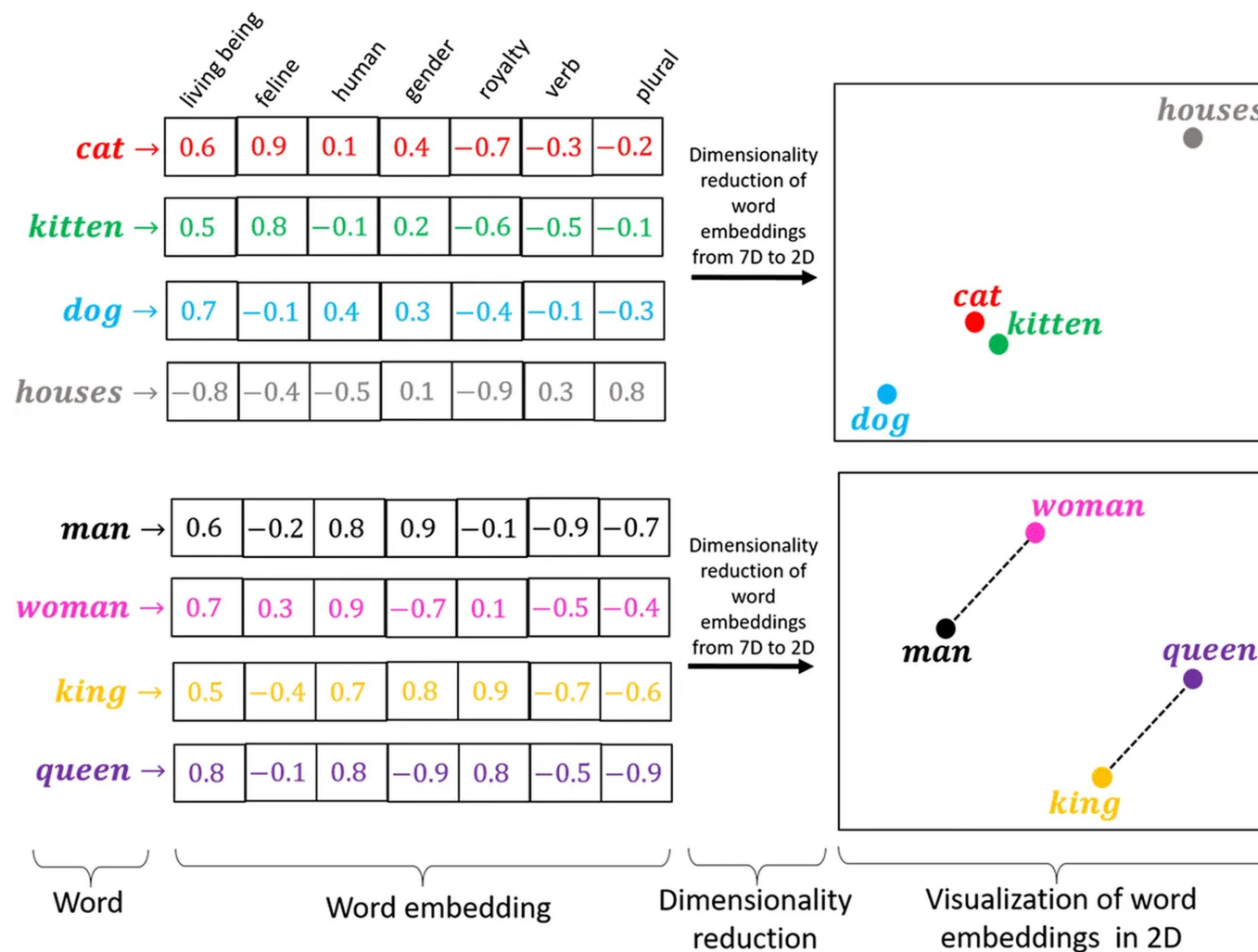


Figure 15: Semantic Word Embeddings and Vector Space Relationships: Mapping discrete words into dense continuous feature vectors captures semantic attributes and linear geometric relationships (e.g., $\vec{man} - \vec{woman} \approx \vec{king} - \vec{queen}$), enabling 2D projection and semantic clustering for LLM retrieval. (Source: <https://medium.com/@hari4om/word-embedding-d816f643140>)

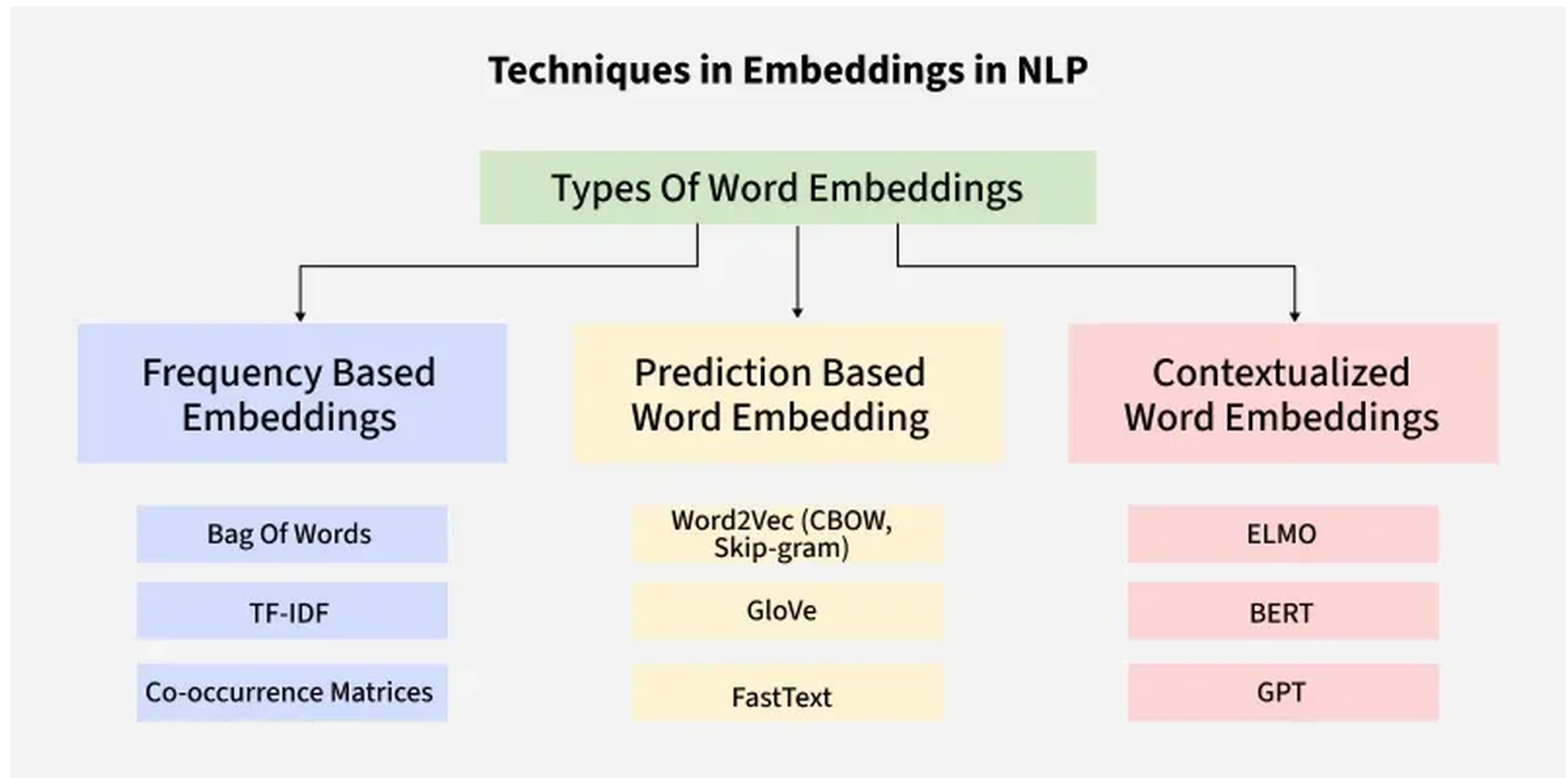


Figure 16: Taxonomy of Word Embedding Techniques in NLP: Categorization across Frequency-based models (Bag of Words, TF-IDF), Prediction-based static embeddings (Word2Vec, GloVe, FastText), and Deep Contextualized neural representations (ELMo, BERT, GPT).
(Source: <https://www.geeksforgeeks.org/nlp/word-embeddings-in-nlp/>)

1.2.5.4 Python TensorFlow Implementation: MNIST 2D Autoencoder vs. Linear PCA

To empirically compare linear orthogonal subspace projections against non-linear deep latent manifolds, we implement an end-to-end benchmark on the **MNIST handwritten digits dataset** ($28 \times 28 = 784$ pixels):

- **Problem Formulation:** Compress each 784-dimensional digit vector $\mathbf{x} \in [0, 1]^{784}$ down to an extreme two-dimensional latent bottleneck $\mathbf{z} = [z_1, z_2]^T \in \mathbb{R}^2$, and then reconstruct $\hat{\mathbf{x}} \approx \mathbf{x}$.
- **Linear Baseline (PCA):** Computes the analytical orthogonal projection $\mathbf{z}_{\text{PCA}} = (\mathbf{x} - \boldsymbol{\mu})\mathbf{V}_2$, reconstructing via $\hat{\mathbf{x}}_{\text{PCA}} = \mathbf{z}_{\text{PCA}}\mathbf{V}_2^T + \boldsymbol{\mu}$.
- **Deep Non-Linear Autoencoder:** Employs a symmetric Multi-Layer Perceptron (MLP) architecture:

$$\begin{aligned} \text{Encoder: } & 784 \xrightarrow{\text{ReLU}} 512 \xrightarrow{\text{ReLU}} 256 \xrightarrow{\text{ReLU}} 64 \xrightarrow{\text{Linear}} 2 \\ \text{Decoder: } & 2 \xrightarrow{\text{ReLU}} 64 \xrightarrow{\text{ReLU}} 256 \xrightarrow{\text{ReLU}} 512 \xrightarrow{\text{Sigmoid}} 784 \end{aligned}$$

- **Optimization, Regularization & Early Stopping:** Trained using the **AdamW optimizer** with standard learning rate $\eta = 10^{-3}$ and weight decay $\lambda = 10^{-4}$ (upper limit of 1000 epochs). An **Early Stopping** callback monitors validation loss (patience = 5), terminating optimization and restoring the optimal weights once generalization error ceases to improve.

$$\mathcal{L}_{\text{MSE}}(\mathbf{x}, \hat{\mathbf{x}}) = \frac{1}{784} \sum_{j=1}^{784} (x_j - \hat{x}_j)^2$$

- **Latent Centroid Prototype Decoding:** In addition to sample reconstruction, we calculate the empirical class centroids in 2D latent space from the training set $\mathbf{c}_d = \mathbb{E}[\mathbf{z}_{\text{train}} \mid y_{\text{train}} = d] \in \mathbb{R}^2$ for each digit class $d \in \{0, \dots, 9\}$ and pass them through the Decoder to generate canonical noise-free prototypes $\hat{\mathbf{x}}_{\text{centroid},d} = \text{Decoder}(\mathbf{c}_d)$.

1.2.6 Python Implementation: TensorFlow/Keras, PCA Baseline & Visualization

```
import os
import sys
import random
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
import tensorflow as tf
from tensorflow.keras import layers, models

# Set fixed random seeds for deterministic reproducibility
os.environ['PYTHONHASHSEED'] = '42'
random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)
tf.keras.utils.set_random_seed(42)

script_dir = os.path.dirname(os.path.abspath(__file__))
out_dir = os.path.abspath(os.path.join(script_dir, ".."))
os.makedirs(out_dir, exist_ok=True)

plt.rcParams['font.sans-serif'] = 'Helvetica', 'DejaVu Sans', 'Arial'
plt.rcParams['mathtext.fontset'] = 'cm'

print("=" * 70, flush=True)
print("  STEP 1: LOAD & PREPROCESS MNIST DATASET (28x28 Grayscale Digits)", flush=True)
print("=" * 70, flush=True)
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Normalize pixel intensities to [0, 1] and flatten (28x28 -> 784)
x_train = x_train.astype(np.float32) / 255.0
x_test = x_test.astype(np.float32) / 255.0
x_train_flat = x_train.reshape(-1, 784)
x_test_flat = x_test.reshape(-1, 784)

print(f"Train samples: {x_train_flat.shape[0]} | Test samples: {x_test_flat.shape[0]}", flush=True)
print(f"Input dimension: {x_train_flat.shape[1]} (28x28 pixels) -> Latent bottleneck z: 2 dimensions\n", flush=True)
```

```
print("=" * 70, flush=True)
print("  STEP 2: LINEAR BASELINE - 2D PRINCIPAL COMPONENT ANALYSIS (PCA)", flush=True)
print("=" * 70, flush=True)
pca = PCA(n_components=2, random_state=42)
z_train_pca = pca.fit_transform(x_train_flat)
z_test_pca = pca.transform(x_test_flat)

# Reconstruct train digits from 2D linear projection
x_train_pca_recon = pca.inverse_transform(z_train_pca)
pca_mse_train = float(np.mean((x_train_flat - x_train_pca_recon) ** 2))

# Compute PCA Centroids from training data for each class
pca_centroids = np.zeros((10, 2), dtype=np.float32)
for d in range(10):
    pca_centroids[d] = np.mean(z_train_pca[y_train == d], axis=0)

print(f"PCA Total Explained Variance Ratio (k=2): {np.sum(pca.explained_variance_ratio_)*100:.2f}%", flush=True)
print(f"  - PC1 Explained Variance: {pca.explained_variance_ratio_[0]*100:.2f}%", flush=True)
print(f"  - PC2 Explained Variance: {pca.explained_variance_ratio_[1]*100:.2f}%", flush=True)
print(f"PCA Train Reconstruction MSE: {pca_mse_train:.5f}\n", flush=True)

print("=" * 70, flush=True)
print("  STEP 3: DEEP NON-LINEAR AUTOENCODER (1000 EPOCHS LIMIT & EARLY STOPPING)", flush=True)
print("=" * 70, flush=True)
# Modular Encoder and Decoder definition
# 1. Encoder (Deep Non-Linear MLP)
encoder_input = layers.Input(shape=(784,), name="encoder_input")
e = layers.Dense(512, activation='relu', name="enc_dense_1")(encoder_input)
e = layers.Dense(256, activation='relu', name="enc_dense_2")(e)
e = layers.Dense(64, activation='relu', name="enc_dense_3")(e)
z_bottleneck = layers.Dense(2, activation='linear', name="bottleneck_z")(e)
encoder = models.Model(encoder_input, z_bottleneck, name="Encoder_2D")

# 2. Decoder
decoder_input = layers.Input(shape=(2,), name="decoder_input")
d = layers.Dense(64, activation='relu', name="dec_dense_1")(decoder_input)
d = layers.Dense(256, activation='relu', name="dec_dense_2")(d)
d = layers.Dense(512, activation='relu', name="dec_dense_3")(d)
```

```
decoded_output = layers.Dense(784, activation='sigmoid', name="dec_output")(d)
decoder = models.Model(decoder_input, decoded_output, name="Decoder_2D")

# 3. Combined Autoencoder
autoencoder_output = decoder(encoder(encoder_input))
autoencoder = models.Model(encoder_input, autoencoder_output, name="Deep_Autoencoder_2D")

# Standard learning rate (1e-3) with weight decay (1e-4) via AdamW
optimizer = tf.keras.optimizers.AdamW(
    learning_rate=1e-3,
    weight_decay=1e-4
)
autoencoder.compile(
    optimizer=optimizer,
    loss='mse'
)
print('Model: "Deep_Autoencoder_2D" (784 -> 512 -> 256 -> 64 -> 2 -> 64 -> 256 -> 512 -> 784)', flush=True)
print(f"Total Trainable Parameters: {autoencoder.count_params():,} (4.20 MB)\n", flush=True)

# Early Stopping Callback: Stop when val_loss starts going up / ceases improving
early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=5,
    min_delta=1e-5,
    restore_best_weights=True,
    verbose=0
)

# Custom Epoch Logger Callback
class EpochLogger(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs=None):
        ep = epoch + 1
        if ep == 1 or ep % 25 == 0:
            val_str = f" - val_loss: {logs['val_loss']:.5f}" if 'val_loss' in logs else ""
            print(f"Epoch {ep:04d}/1000 - loss: {logs['loss']:.5f}{val_str}", flush=True)
    def on_train_end(self, logs=None):
        best_ep = self.model.history.epoch[-1] + 1
        print(f"Training stopped at epoch {best_ep:04d}/1000 (Early Stopping restored best weights)", flush=True)
```

```
# Train model with limit 1000 epochs
epochs = 1000
batch_size = 512
history = autoencoder.fit(
    x_train_flat, x_train_flat,
    epochs=epochs,
    batch_size=batch_size,
    shuffle=True,
    validation_data=(x_test_flat, x_test_flat),
    validation_batch_size=2048,
    verbose=0,
    callbacks=[early_stopping, EpochLogger()]
)

# Predict Latent Code & Reconstructed Output on Training Data
z_train_ae = encoder.predict(x_train_flat, batch_size=1024, verbose=0)
x_train_ae_recon = autoencoder.predict(x_train_flat, batch_size=1024, verbose=0)
ae_mse_train = float(np.mean((x_train_flat - x_train_ae_recon) ** 2))

# Compute 2D Latent Centroids from Training Data for each class (c_d = E[z | y = d]) and decode
ae_centroids = np.zeros((10, 2), dtype=np.float32)
centroid_decoded_digits = np.zeros((10, 784), dtype=np.float32)

for d in range(10):
    mask = (y_train == d)
    ae_centroids[d] = np.mean(z_train_ae[mask], axis=0)
    centroid_decoded_digits[d] = decoder.predict(ae_centroids[d:d+1], verbose=0)[0]

print("\n" + "=" * 70, flush=True)
print(" STEP 4: NUMERICAL COMPARISON & RECONSTRUCTION EVALUATION", flush=True)
print("=" * 70, flush=True)
print(f"Linear PCA (z=2) Train MSE:      {pca_mse_train:.5f}", flush=True)
print(f"Deep Autoencoder (z=2) Train MSE: {ae_mse_train:.5f}", flush=True)
improvement = ((pca_mse_train - ae_mse_train) / pca_mse_train) * 100
print(f"Autoencoder Error Reduction:      {improvement:.2f}%\n", flush=True)

for d in range(10):
```



```

    print(f"  Digit {d} Latent Centroid c_{d}: (z1 = {ae_centroids[d, 0]:+7.3f}, z2 = {ae_centroids[d, 1]:+7.3f})", flush=True)

# =====
# FIGURE 1: 4-Row Visual Digit Reconstruction & Centroid Decoding Benchmark
# Row 1: Original Train Digit Sample (Ground Truth)
# Row 2: Linear PCA Reconstruction (z=2)
# Row 3: Deep Autoencoder Reconstruction (z=2)
# Row 4: Autoencoder Latent Centroid Prototype Decoding (c_d in R^2)
# =====
digit_indices = []
for d in range(10):
    match_idx = np.where(y_train == d)[0][0]
    digit_indices.append(match_idx)

fig, axes = plt.subplots(4, 10, figsize=(20, 11.2), dpi=300,
                        gridspec_kw={'hspace': 0.70, 'wspace': 0.12})

row_labels = [
    "Original Sample (784-dim)\nGround Truth Train Image",
    f"Linear PCA (z=2)\nReconstruction (MSE: {pca_mse_train:.4f})",
    f"Deep Autoencoder (z=2)\nSample Recon (MSE: {ae_mse_train:.4f})",
    "Latent Centroid ( $\mathbf{c}_d \in \mathbb{R}^2$ )\nDecoded Mean Prototype"
]
row_colors = ['#0F172A', '#B91C1C', '#15803D', '#4338CA']

for r in range(4):
    for c in range(10):
        idx = digit_indices[c]
        ax = axes[r, c]

        if r == 0:
            ax.imshow(x_train[idx], cmap='gray')
            ax.set_title(f"Digit {c}", fontsize=13, fontweight='bold', pad=8, color='#334155')
        elif r == 1:
            ax.imshow(x_train_pca_recon[idx].reshape(28, 28), cmap='gray')
        elif r == 2:
            ax.imshow(x_train_ae_recon[idx].reshape(28, 28), cmap='gray')
        else:

```

```

        ax.imshow(centroid_decoded_digits[c].reshape(28, 28), cmap='gray')

    ax.set_xticks([])
    ax.set_yticks([])

    # Add clear left-hand row header label on the first column
    if c == 0:
        ax.set_ylabel(row_labels[r], fontsize=12.5, fontweight='bold',
                      color=row_colors[r], labelpad=12, multialignment='center')

fig.suptitle("MNIST 28x28 Reconstruction Benchmark: 2D Linear PCA vs. Deep Autoencoder & Latent Centroid Decoding",
            fontsize=16, fontweight='bold', y=0.985, color='#0F172A')

fig1_path = "section/ch06_figures/fig_6_02j_mnist_reconstruction_ae_vs_pca.png" if os.path.exists("section") else "
fig_6_02j_mnist_reconstruction_ae_vs_pca.png"
if os.path.dirname(fig1_path):
    os.makedirs(os.path.dirname(fig1_path), exist_ok=True)
plt.savefig(fig1_path, dpi=300, bbox_inches='tight', pad_inches=0.1)
plt.close()
print(f"\nSaved: {fig1_path}", flush=True)

# =====
# FIGURE 2: 2D Latent Embedding Space Colored by Digit Class with Centroids
# =====
fig, axes = plt.subplots(1, 2, figsize=(18, 8.2), dpi=300)

n_scatter = 5000 # Scatter 5,000 training points
cmap = plt.colormaps['tab10'].resampled(10)

# Panel (a): PCA 2D Projection
scatter1 = axes[0].scatter(z_train_pca[:n_scatter, 0], z_train_pca[:n_scatter, 1],
                          c=y_train[:n_scatter], cmap=cmap, alpha=0.55, s=12, edgecolors='none')
axes[0].scatter(pca_centroids[:, 0], pca_centroids[:, 1],
               color='black', marker='X', s=90, edgecolors='white', linewidth=1.2, zorder=5, label='Class Centroids')
for d in range(10):
    axes[0].annotate(f"{d}", (pca_centroids[d, 0], pca_centroids[d, 1]),
                    fontsize=10, fontweight='bold', color='black',
                    bbox=dict(boxstyle="round,pad=0.2", fc="white", ec="black", alpha=0.85, lw=1))

```

```

axes[0].set_title("(a) Linear PCA 2D Latent Space ($z_1$ vs. $z_2$)\nHigh Overlap Between Digit Classes",
                  fontsize=13.5, fontweight='bold', pad=10, color='#0F172A')
axes[0].set_xlabel("Principal Component 1 ($PC_1$)", fontsize=12, fontweight='bold')
axes[0].set_ylabel("Principal Component 2 ($PC_2$)", fontsize=12, fontweight='bold')
axes[0].grid(True, linestyle='--', alpha=0.35)

# Panel (b): Autoencoder 2D Bottleneck Space with Centroids
scatter2 = axes[1].scatter(z_train_ae[:n_scatter, 0], z_train_ae[:n_scatter, 1],
                          c=y_train[:n_scatter], cmap=cmap, alpha=0.55, s=12, edgecolors='none')
axes[1].scatter(ae_centroids[:, 0], ae_centroids[:, 1],
                color='black', marker='X', s=90, edgecolors='white', linewidth=1.2, zorder=5, label='Latent Centroids $\mathbf{c}_d$')
for d in range(10):
    axes[1].annotate(f"$\mathbf{c}_{{d}}$", (ae_centroids[d, 0], ae_centroids[d, 1]),
                    fontsize=10, fontweight='bold', color='#1E1B4B',
                    bbox=dict(boxstyle="round,pad=0.25", fc="#EEF2FF", ec="#4338CA", alpha=0.9, lw=1.2))

axes[1].set_title("(b) Deep Autoencoder 2D Bottleneck ($z_1$ vs. $z_2$)\nNon-Linear Semantic Clustering with Centroids $\mathbf{c}_d$",
                  ,
                  fontsize=13.5, fontweight='bold', pad=10, color='#0F172A')
axes[1].set_xlabel("Latent Bottleneck Dimension 1 ($z_1$)", fontsize=12, fontweight='bold')
axes[1].set_ylabel("Latent Bottleneck Dimension 2 ($z_2$)", fontsize=12, fontweight='bold')
axes[1].grid(True, linestyle='--', alpha=0.35)

# Colorbar for digit classes
cbar = fig.colorbar(scatter2, ax=axes, orientation='horizontal', fraction=0.06, pad=0.14, ticks=range(10))
cbar.set_label("MNIST Digit Class Label (0-9)", fontsize=13, fontweight='bold', labelpad=6)
cbar.set_ticklabels([f"Digit {i}" for i in range(10)], fontsize=11, fontweight='bold')

fig.suptitle("2D Latent Embedding Manifold Comparison & Class Centroids: PCA vs. Deep Autoencoder ($z \in \mathbb{R}^2$)",
             fontsize=15.5, fontweight='bold', y=0.98, color='#0F172A')

fig2_path = "section/ch06_figures/fig_6_02k_mnist_2d_latent_space_ae_vs_pca.png" if os.path.exists("section") else "
fig_6_02k_mnist_2d_latent_space_ae_vs_pca.png"
if os.path.isdir(fig2_path):
    os.makedirs(os.path.dirname(fig2_path), exist_ok=True)
plt.savefig(fig2_path, dpi=300, bbox_inches='tight', pad_inches=0.1)
plt.close()

```

```
print(f"Saved: {fig2_path}", flush=True)

print("\nAll tasks completed successfully!", flush=True)
```

1.2.6.1 Execution Logs: Training Progression & Evaluation Output

```
=====
STEP 1: LOAD & PREPROCESS MNIST DATASET (28x28 Grayscale Digits)
=====
Train samples: 60000 | Test samples: 10000
Input dimension: 784 (28x28 pixels) -> Latent bottleneck z: 2 dimensions

=====
STEP 2: LINEAR BASELINE - 2D PRINCIPAL COMPONENT ANALYSIS (PCA)
=====
PCA Total Explained Variance Ratio (k=2): 16.80%
- PC1 Explained Variance: 9.70%
- PC2 Explained Variance: 7.10%
PCA Train Reconstruction MSE: 0.05595

=====
STEP 3: DEEP NON-LINEAR AUTOENCODER (1000 EPOCHS LIMIT & EARLY STOPPING)
=====
Model: "Deep_Autoencoder_2D" (784 -> 512 -> 256 -> 64 -> 2 -> 64 -> 256 -> 512 -> 784)
Total Trainable Parameters: 1,100,434 (4.20 MB)

Epoch 0001/1000 - loss: 0.07802 - val_loss: 0.05883
Epoch 0025/1000 - loss: 0.03572 - val_loss: 0.03591
Epoch 0050/1000 - loss: 0.03328 - val_loss: 0.03440
Epoch 0075/1000 - loss: 0.03205 - val_loss: 0.03360
Training stopped at epoch 0088/1000 (Early Stopping restored best weights)

=====
```

```
STEP 4: NUMERICAL COMPARISON & RECONSTRUCTION EVALUATION
=====
Linear PCA (z=2) Train MSE:      0.05595
Deep Autoencoder (z=2) Train MSE: 0.03164
Autoencoder Error Reduction:     43.46%

Digit 0 Latent Centroid c_0: (z1 = -0.120, z2 = -6.473)
Digit 1 Latent Centroid c_1: (z1 = +20.078, z2 = +29.025)
Digit 2 Latent Centroid c_2: (z1 = -3.299, z2 = +13.772)
Digit 3 Latent Centroid c_3: (z1 = +0.788, z2 = +1.158)
Digit 4 Latent Centroid c_4: (z1 = +14.779, z2 = +0.329)
Digit 5 Latent Centroid c_5: (z1 = +2.165, z2 = -1.584)
Digit 6 Latent Centroid c_6: (z1 = +14.557, z2 = -6.939)
Digit 7 Latent Centroid c_7: (z1 = +37.640, z2 = +4.540)
Digit 8 Latent Centroid c_8: (z1 = +2.678, z2 = +0.815)
Digit 9 Latent Centroid c_9: (z1 = +15.788, z2 = +0.637)

Saved: section/ch06_figures/fig_6_02j_mnist_reconstruction_ae_vs_pca.png
Saved: section/ch06_figures/fig_6_02k_mnist_2d_latent_space_ae_vs_pca.png
All tasks completed successfully!
```

MNIST 28x28 Reconstruction Benchmark: 2D Linear PCA vs. Deep Autoencoder & Latent Centroid Decoding

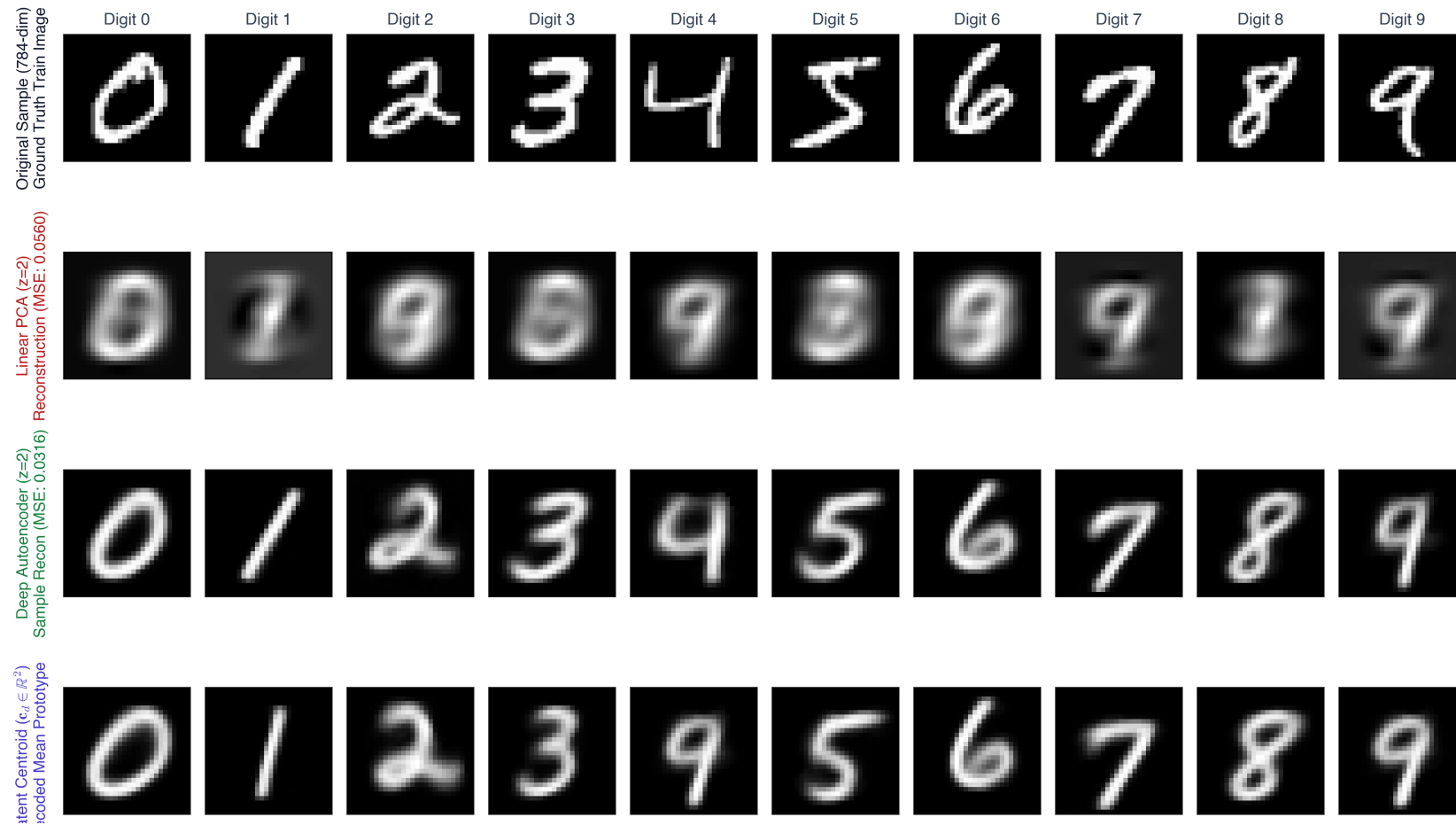


Figure 17: Visual reconstruction and prototype decoding benchmark across 10 representative MNIST training digits (0 through 9): Row 1 shows ground truth original 28×28 training images (784 dimensions); Row 2 shows linear PCA reconstruction ($z = 2$, $\text{MSE} = 0.0560$); Row 3 shows deep non-linear Autoencoder reconstruction ($z = 2$, $\text{MSE} = 0.0316$); Row 4 displays the decoded prototype from class latent centroids ($\mathbf{c}_d = \mathbb{E}[\mathbf{z} \mid y = d] \in \mathbb{R}^2$), synthesizing an idealized, canonical prototype for each digit class. The Autoencoder with Early Stopping achieves a 43.46% reduction in reconstruction error over PCA.

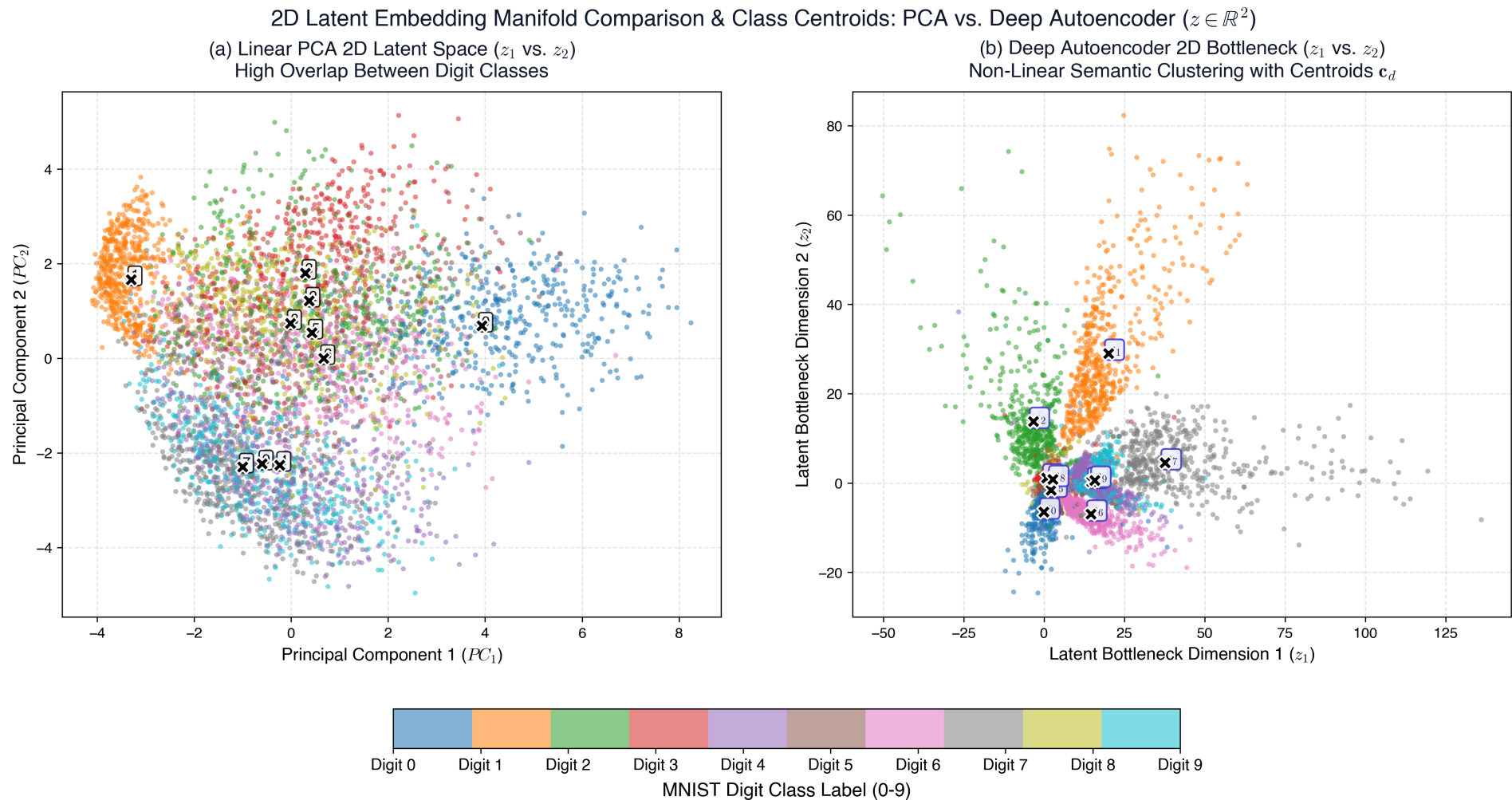


Figure 18: 2D Latent space embedding comparison (z_1 vs. z_2) on training samples colored by ground-truth digit class (0–9) with class centroids $\mathbf{c}_d$: (a) Linear PCA projects data onto orthogonal variance axes, resulting in a single heavily overlapped cloud; (b) Deep Autoencoder non-linearly unfolds the latent manifold, organizing distinct digit classes into separate semantic clusters and trajectory branches with annotated centroid coordinates $\mathbf{c}_0, \dots, \mathbf{c}_9$.

Linear PCA vs. Non-Linear Autoencoders

- **PCA:** Uses linear projections, which may not capture the complex non-linear structure of image data well when the latent dimension is very small ($z = 2$).
- **Autoencoder:** Uses non-linear neural network layers (ReLU on encoder and decoder) to learn a more flexible representation and can achieve better reconstruction with a low-dimensional latent space.
- **Latent Space:** The learned latent representation can capture meaningful patterns in the data, allowing similar samples to form meaningful groups.
- **Continuous Latent Geometry & Centroid Decoding:**
Passing the class centroids through the Decoder produces crisp canonical digit prototypes, proving that the Autoencoder latent space forms a continuous semantic manifold capable of smooth feature representation and interpolation.

1.3 Generative Adversarial Networks (Data Generation)

1.3.1 The Basic Idea

A **Generative Adversarial Network (GAN)**², introduced in 2014, is a generative model that learns to create realistic synthetic data through a **two-player competitive game** between two neural networks:

- **The Generator (G):** The *Counterfeiter*. It takes a random noise vector $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ as input and transforms it into a synthetic sample:

$$\mathbf{x}_{\text{fake}} = G(\mathbf{z}).$$

The Generator's goal is to produce samples that are sufficiently realistic to fool the Discriminator.

- **The Discriminator (D):** The *Detective*. It receives both real samples from the training dataset and synthetic samples produced by the Generator. It outputs a score

$$D(\mathbf{x}) \in [0, 1],$$

representing how likely the input is to be a real sample. Its goal is to distinguish real samples from generated samples.

The two networks are trained simultaneously but have opposing objectives: the **Generator tries to fool the Discriminator**, while the **Discriminator tries to correctly distinguish real samples from fake samples**. Through this competition, the Generator gradually learns to produce increasingly realistic data.

²The word **Adversarial** (ปฏิปักษ์ หรือ ประปักษ์) comes from “adversary” (an opponent, rival, or competing party).

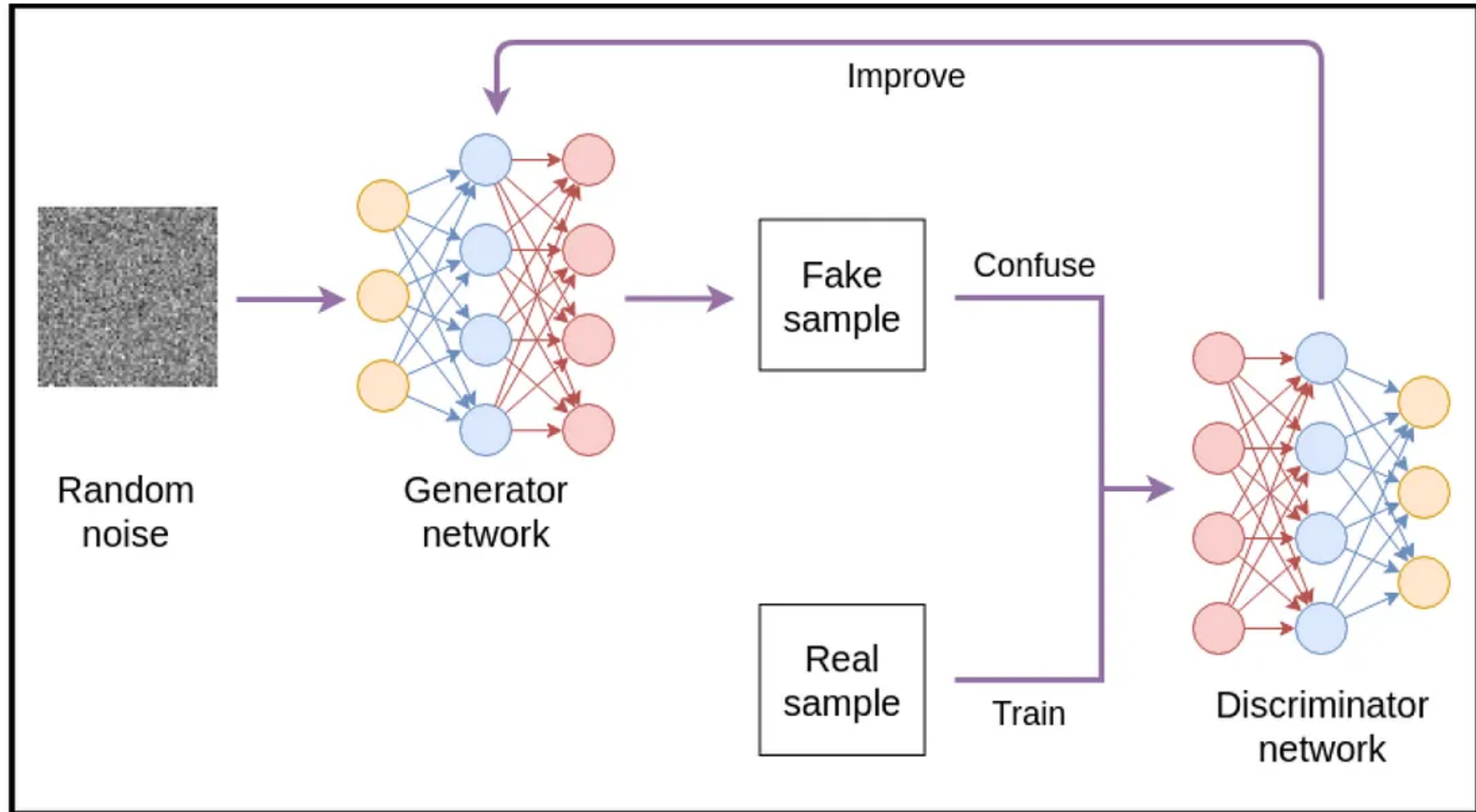


Figure 19: Generative Adversarial Network (GAN) Interaction Flow: The Generator network maps random input noise into synthetic (fake) samples aimed at confusing the Discriminator, while real samples from the dataset are provided to train the Discriminator to distinguish authentic data from generated fakes. Feedback from the Discriminator's assessment is passed back to improve the Generator over successive training iterations. (Source:

<https://medium.com/@hanen23/generative-ia-introduction-to-generative-adversarial-network-gan-7027e94d69ab>)

Generative Adversarial Network (GAN): Two-Network Minimax Game

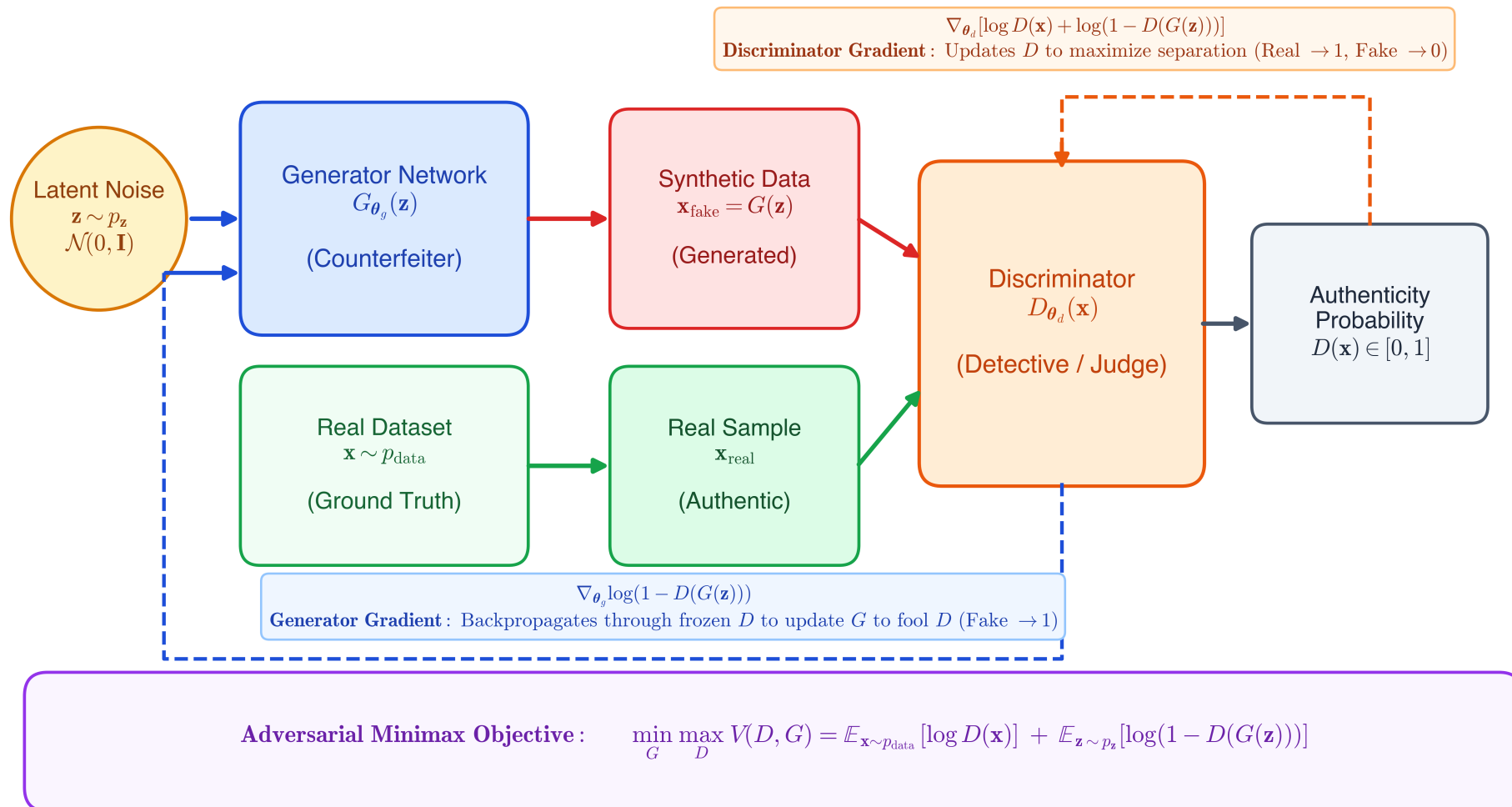


Figure 20: The Generative Adversarial Network (GAN) architecture: Generator G synthesizing candidate fakes from noise $\mathbf{z}$ while Discriminator D scores authenticity, forming an adversarial minimax optimization loop.

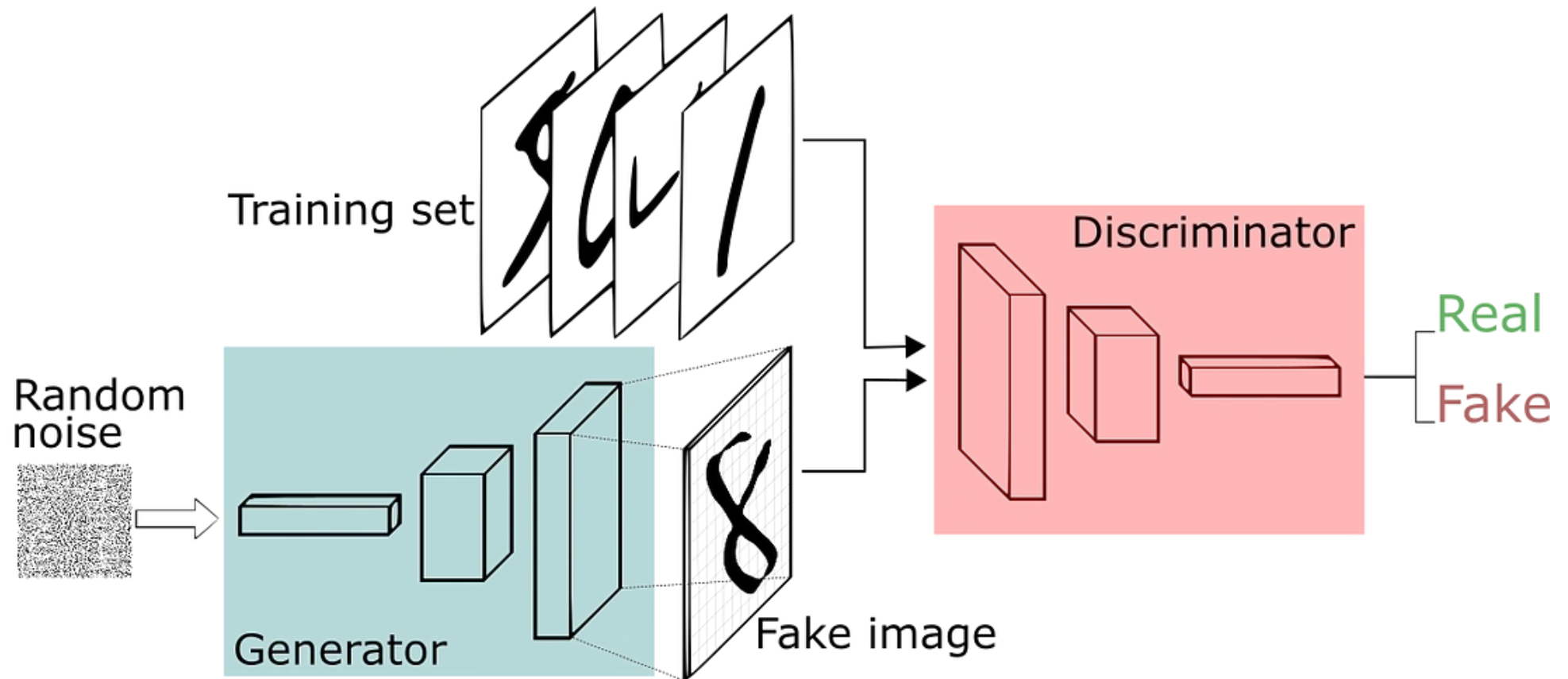


Figure 21: Visual overview of GAN data flow and training pipeline: The Generator network transforms random noise into synthetic samples (e.g., candidate handwritten digits), while the Discriminator network evaluates authentic training set images alongside synthetic candidates to classify each as Real or Fake. (Source: <https://medium.com/@nutorbitx/gans-%E0%B8%AD%E0%B8%B0%E0%B9%84%E0%B8%A3%E0%B8%84%E0%B8%B7%E0%B8%AD-generative-adversarial-networks-7973ae70db70>)

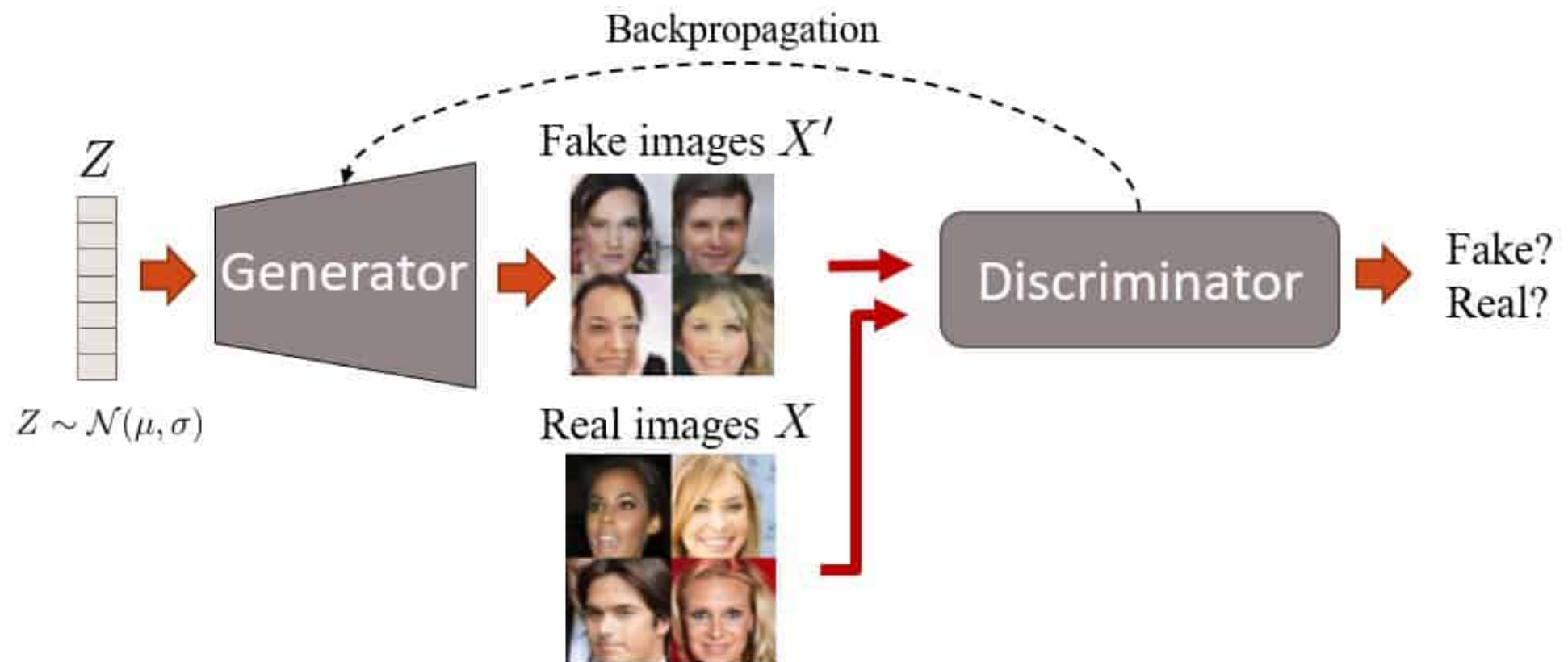


Figure 22: End-to-End GAN Training Overview with Dual Backpropagation Paths: (1) **Discriminator Training:** The Discriminator functions as a standard binary classifier trained via traditional backpropagation (using binary cross-entropy loss) to classify real images X as real (label 1) and generated images X' as fake (label 0), updating its weights just like a conventional classification network; (2) **Generator Training:** The Generator receives error feedback backpropagated all the way from the Discriminator output through the frozen Discriminator layers, updating the Generator weights to transform random noise vectors $Z \sim \mathcal{N}(\mu, \sigma)$ into increasingly realistic synthetic images X' that fool the Discriminator. (Source: <https://learnopencv.com/introduction-to-generative-adversarial-networks/>)

1.3.2 Why We Need GANs

A GAN addresses this challenge through the interaction between two neural networks: a **Generator** and a **Discriminator**.

The **Generator** creates synthetic samples, while the **Discriminator** evaluates whether a sample is real or generated. The feedback from the Discriminator helps the Generator gradually improve the quality of its outputs.

During training:

Generator improves its ability to create realistic fakes

Discriminator improves its ability to detect fakes

As results, the Generator becomes better at producing realistic samples, while the Discriminator becomes better at detecting generated samples.

Thus, the key idea of a GAN is:

Generator creates $\longleftrightarrow$ Discriminator evaluates

Through this adversarial process, GANs can generate realistic images, faces, and other types of synthetic data.

1.3.3 How GANs Learn: Adversarial Optimization

1.3.3.1 Minimax Optimization

The Generator G and Discriminator D are trained through an adversarial game (เกมที่มีการแข่งขันกัน). The Discriminator tries to distinguish real samples from generated samples, while the Generator tries to produce samples that the Discriminator cannot distinguish from real data.

The original GAN objective is:

$$\min_G \max_D V(D, G) = \underbrace{\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D(\mathbf{x})]}_{\substack{\text{Expect } D(\mathbf{x})=1 \text{ (real)} \\ \log(1)=0}} + \underbrace{\mathbb{E}_{\mathbf{z} \sim p_z} [\log (1 - D(G(\mathbf{z})))]}_{\substack{\text{Expect } D(G(\mathbf{z}))=0 \text{ (fake)} \\ \log(1-0)=\log(1)=0}} \quad (6)$$

Variables:

- $\mathbf{x} \sim p_{\text{data}}$: Real data sample from the training data distribution.
- $\mathbf{z} \sim p_z$: Random noise vector sampled from the latent noise distribution.
- $G(\mathbf{z})$: Synthetic sample generated by the Generator G .
- $D(\mathbf{x})$: Discriminator's Probability that $\mathbf{x}$ is real, $D(\mathbf{x}) \in [0, 1]$; 1 = real, 0 = fake.
- $D(G(\mathbf{z}))$: Probability that the generated sample $G(\mathbf{z})$ is real, $D(G(\mathbf{z})) \in [0, 1]$; 1 = real, 0 = fake.
- $V(D, G)$: The **value function** of the adversarial game, which measures the overall performance of the Generator–Discriminator interaction.

The Training Process:

1. Train the Discriminator:

Keep G fixed and train D using real samples and generated samples. The Discriminator learns to classify real samples as 1 and generated samples as 0.

2. Train the Generator:

Keep D fixed and update G so that its generated samples are more likely to be classified as real by D .

3. Adversarial Training:

These two steps are **repeated alternately**. As D improves at detecting generated samples, G receives stronger feedback to improve its outputs.

Ideally, training reaches a point where the Discriminator can no longer tell whether a sample is real or generated. At this point, it gives a probability of about

$$D(\mathbf{x}) \approx 0.5$$

for both real and generated samples.

Simple meaning:

The Generator becomes good enough that the Discriminator cannot tell real and fake samples apart.

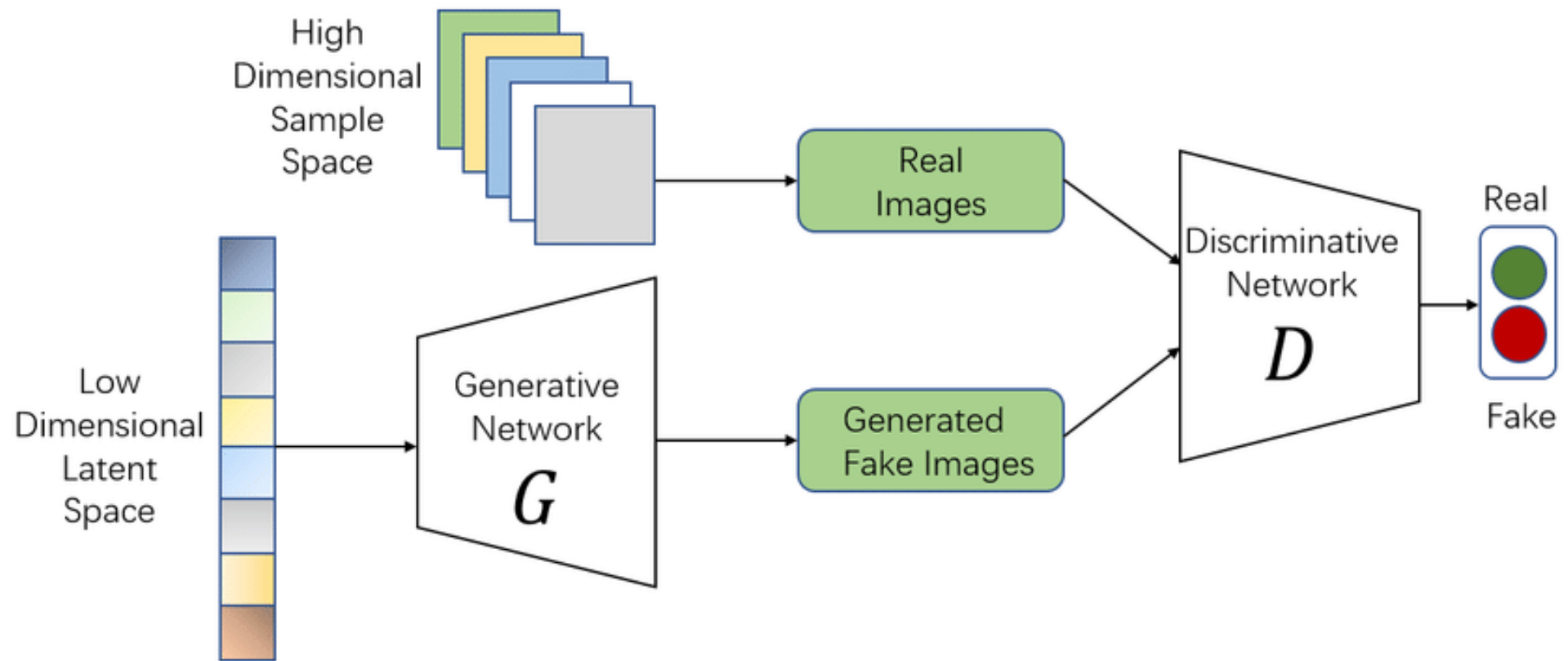


Figure 23: Vanilla GAN Architecture: Mapping from a low-dimensional latent noise space via the Generative Network G to produce synthetic candidates, while the Discriminative Network D evaluates candidate images against the high-dimensional real sample space to output a Real vs. Fake decision. (Source: <https://www.labellerr.com/blog/what-is-gan-how-does-it-work/>)

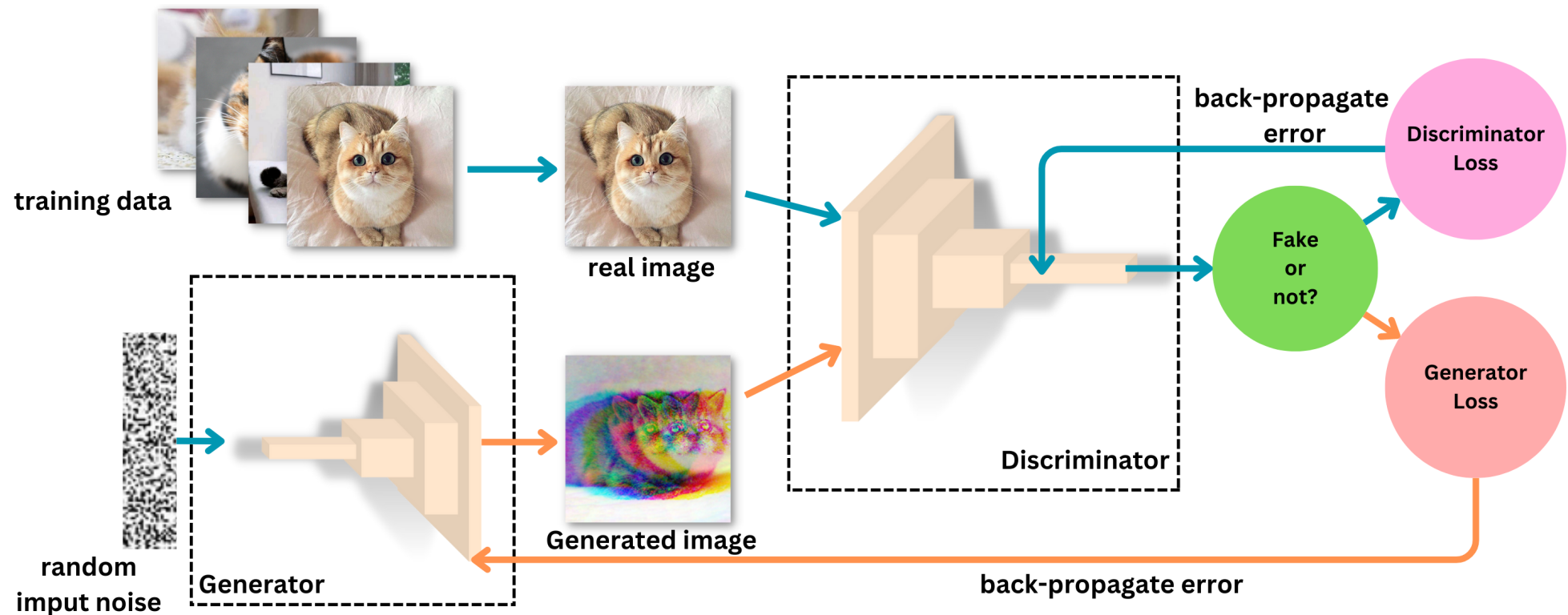


Figure 24: Adversarial Training Dynamics and Dual Loss Backpropagation in GANs: Random noise is upsampled through Generator transpose convolutional layers to produce synthetic outputs, while the Discriminator processes real training images and generated samples to compute separate Discriminator Loss (optimizing classification accuracy) and Generator Loss (backpropagating through the frozen Discriminator to improve generative quality). (Source: <https://newsletter.theaiedge.io/p/how-generative-adversarial-networks>)

1.3.4 PyTorch Implementation: DCGAN on Aligned Human Faces

1.3.4.1 Deep Convolutional GAN (DCGAN) Architecture

To demonstrate generative adversarial learning in practice, we implement a classic **Deep Convolutional GAN (DCGAN)** (Radford et al., 2015) in PyTorch trained on 64×64 RGB images from an aligned human face dataset (13,233 samples expanded to 26,466 training samples via horizontal flip augmentation).

- **Generator (G):** Projects a 100-dimensional standard Gaussian latent vector $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{100})$ through 5 transposed convolutional stages (`nn.ConvTranspose2d`³) with Batch Normalization and ReLU activations ($100 \times 1 \times 1 \rightarrow 512 \times 4 \times 4 \rightarrow 256 \times 8 \times 8 \rightarrow 128 \times 16 \times 16 \rightarrow 64 \times 32 \times 32 \rightarrow 3 \times 64 \times 64$), outputting a $64 \times 64 \times 3$ RGB image via $\tanh(\cdot) \in [-1, 1]$. Total parameters: 3,576,704.
- **Discriminator (D):** Processes $64 \times 64 \times 3$ candidate images through 5 strided downsampling convolutions with LeakyReLU (0.2) activations and Batch Normalization ($3 \times 64 \times 64 \rightarrow 64 \times 32 \times 32 \rightarrow 128 \times 16 \times 16 \rightarrow 256 \times 8 \times 8 \rightarrow 512 \times 4 \times 4 \rightarrow 1 \times 1 \times 1$), outputting an authenticity probability score $D(\mathbf{x}) \in [0, 1]$ via $\sigma(\cdot)$. Total parameters: 2,765,568.

³`nn.ConvTranspose2d` (often referred to as *transposed convolution* or *fractionally-strided convolution*) acts as the spatial inverse of a standard convolution. While a standard convolution compresses and downsamples spatial dimensions ($H \times W \rightarrow H/s \times W/s$), a transposed convolution upsamples spatial feature maps ($H \times W \rightarrow sH \times sW$) using learnable kernel weights, enabling the Generator to expand low-dimensional latent vectors into full-resolution images.

1.3.4.2 Complete PyTorch DCGAN Implementation & Publication Visualization

```
import os
import glob
import random
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as transforms

# -----
# 1. Reproducibility & Device Setup
# -----
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device('mps' if torch.backends.mps.is_available() else ('cuda' if torch.cuda.is_available() else 'cpu'))
print(f"Using compute device: {device}", flush=True)

script_dir = os.path.dirname(os.path.abspath(__file__))
out_dir = os.path.abspath(os.path.join(script_dir, ".."))
os.makedirs(out_dir, exist_ok=True)

# -----
# 2. Aligned Face Dataset Loading (LFW / Aligned Human Faces)
# -----
class AlignedFacesDataset(Dataset):
    def __init__(self, img_size=64):
        print("Loading Aligned Human Faces dataset into memory...", flush=True)
```

```
# Look for cached LFW dataset or fetch via sklearn
lfw_dir = os.path.expanduser('~/.scikit_learn_data/lfw_home')
if not os.path.exists(lfw_dir):
    import sklearn.datasets
    sklearn.datasets.fetch_lfw_people(color=True)

jpg_paths = sorted(glob.glob(os.path.join(lfw_dir, '**/*.jpg'), recursive=True))
if len(jpg_paths) == 0:
    raise RuntimeError(f"No face images found in {lfw_dir}")

# Center crop tightly on facial landmarks (eyes, nose, mouth) and resize to 64x64
self.transform_base = transforms.Compose([
    transforms.CenterCrop(140),
    transforms.Resize(img_size),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)) # Map pixels to [-1, 1]
])

tensors = []
for path in jpg_paths:
    img = Image.open(path).convert('RGB')
    tensors.append(self.transform_base(img))

imgs_orig = torch.stack(tensors)
imgs_flip = torch.flip(imgs_orig, dims=[3]) # Horizontal flip augmentation

# 13,233 original + 13,233 horizontally flipped = 26,466 total training images
self.images = torch.cat([imgs_orig, imgs_flip], dim=0)
self.raw_paths = jpg_paths
print(f"Dataset ready: {self.images.shape} ({len(jpg_paths):,} original + {len(jpg_paths):,} flipped)", flush=True)

def __len__(self):
    return len(self.images)

def __getitem__(self, idx):
    return self.images[idx]

dataset = AlignedFacesDataset(img_size=64)
```

```
# -----
# 3. Deep Convolutional GAN (DCGAN) Architecture (Radford et al., 2015)
# -----
NZ = 100
NGF = 64
NDF = 64
NUM_EPOCHS = 1000
BATCH_SIZE = 64
N_BATCHES_PER_EPOCH = 32 # 2,048 samples per epoch -> 32,000 total minimax gradient steps

def weights_init(m):
    classname = m.__class__.__name__
    if classname.find('Conv') != -1:
        nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif classname.find('BatchNorm') != -1:
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)

class Generator(nn.Module):
    """
    DCGAN Generator:
    Projects a 100-dimensional Gaussian latent vector z in  $R^{100}$  through
    5 transposed convolutional stages to synthesize a 64x64x3 RGB human face image.
    """
    def __init__(self, nz=NZ, ngf=NGF):
        super().__init__()
        self.main = nn.Sequential(
            # Stage 1: 100 x 1 x 1 -> 512 x 4 x 4
            nn.ConvTranspose2d(nz, ngf * 8, 4, 1, 0, bias=False),
            nn.BatchNorm2d(ngf * 8),
            nn.ReLU(True),
            # Stage 2: 512 x 4 x 4 -> 256 x 8 x 8
            nn.ConvTranspose2d(ngf * 8, ngf * 4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf * 4),
            nn.ReLU(True),
            # Stage 3: 256 x 8 x 8 -> 128 x 16 x 16
            nn.ConvTranspose2d(ngf * 4, ngf * 2, 4, 2, 1, bias=False),
```

```

        nn.BatchNorm2d(ngf * 2),
        nn.ReLU(True),
        # Stage 4: 128 x 16 x 16 -> 64 x 32 x 32
        nn.ConvTranspose2d(ngf * 2, ngf, 4, 2, 1, bias=False),
        nn.BatchNorm2d(ngf),
        nn.ReLU(True),
        # Stage 5: 64 x 32 x 32 -> 3 x 64 x 64 (tanh maps pixels to [-1, 1])
        nn.ConvTranspose2d(ngf, 3, 4, 2, 1, bias=False),
        nn.Tanh()
    )

def forward(self, z):
    return self.main(z)

class Discriminator(nn.Module):
    """
    DCGAN Discriminator:
    Processes 64x64x3 RGB candidate images through 5 strided downsampling
    convolutions with LeakyReLU (0.2) to predict authenticity probability D(x) in [0, 1].
    """
    def __init__(self, ndf=NDF):
        super().__init__()
        self.main = nn.Sequential(
            # Stage 1: 3 x 64 x 64 -> 64 x 32 x 32
            nn.Conv2d(3, ndf, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            # Stage 2: 64 x 32 x 32 -> 128 x 16 x 16
            nn.Conv2d(ndf, ndf * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 2),
            nn.LeakyReLU(0.2, inplace=True),
            # Stage 3: 128 x 16 x 16 -> 256 x 8 x 8
            nn.Conv2d(ndf * 2, ndf * 4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 4),
            nn.LeakyReLU(0.2, inplace=True),
            # Stage 4: 256 x 8 x 8 -> 512 x 4 x 4
            nn.Conv2d(ndf * 4, ndf * 8, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 8),
            nn.LeakyReLU(0.2, inplace=True),

```

```

        # Stage 5: 512 x 4 x 4 -> 1 x 1 x 1 (sigmoid authenticity probability)
        nn.Conv2d(ndf * 8, 1, 4, 1, 0, bias=False),
        nn.Sigmoid()
    )

    def forward(self, img):
        return self.main(img).view(-1, 1).squeeze(1)

netG = Generator().to(device)
netD = Discriminator().to(device)

netG.apply(weights_init)
netD.apply(weights_init)

g_params = sum(p.numel() for p in netG.parameters() if p.requires_grad)
d_params = sum(p.numel() for p in netD.parameters() if p.requires_grad)
print(f"Generator total parameters: {g_params:}", flush=True)
print(f"Discriminator total parameters: {d_params:}", flush=True)

criterion = nn.BCELoss()

# Two Time-Scale Update Rule (TTUR)
optimizerD = optim.Adam(netD.parameters(), lr=0.00015, betas=(0.5, 0.999))
optimizerG = optim.Adam(netG.parameters(), lr=0.00020, betas=(0.5, 0.999))

all_images = dataset.images.to(device)
num_samples = len(all_images)

# -----
# 4. 5 Fixed Latent Seeds & Checkpoint Evaluation Tracking
# -----
torch.manual_seed(42)
fixed_noise = torch.randn(5, NZ, 1, 1, device=device)

checkpoints_progress = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
target_epochs = {
    0: 0,
    10: int(NUM_EPOCHS * 0.1),

```



```
20: int(NUM_EPOCHS * 0.2),
30: int(NUM_EPOCHS * 0.3),
40: int(NUM_EPOCHS * 0.4),
50: int(NUM_EPOCHS * 0.5),
60: int(NUM_EPOCHS * 0.6),
70: int(NUM_EPOCHS * 0.7),
80: int(NUM_EPOCHS * 0.8),
90: int(NUM_EPOCHS * 0.9),
100: NUM_EPOCHS
}

saved_images_by_pct = {}

# Checkpoint 0% (Untrained model forward pass)
netG.eval()
with torch.no_grad():
    untrained_out = netG(fixed_noise).detach().cpu() # Shape: (5, 3, 64, 64) in [-1, 1]
    untrained_imgs = (untrained_out.permute(0, 2, 3, 1).numpy() * 0.5 + 0.5).clip(0, 1)
    saved_images_by_pct[0] = [untrained_imgs[i] for i in range(5)]
netG.train()

d_losses = []
g_losses = []

print(f"Starting DCGAN training on ALL {num_samples:,} images for {NUM_EPOCHS} epochs on {device}...", flush=True)
for epoch in range(1, NUM_EPOCHS + 1):
    epoch_d_loss = 0.0
    epoch_g_loss = 0.0

    for b in range(N_BATCHES_PER_EPOCH):
        idx_batch = torch.randint(0, num_samples, (BATCH_SIZE,), device=device)
        real_data = all_images[idx_batch]
        b_size = real_data.size(0)

        # (1) Update Discriminator: maximize log(D(x)) + log(1 - D(G(z)))
        netD.zero_grad(set_to_none=True)
        label_real = torch.full((b_size,), 0.9, dtype=torch.float, device=device) # One-sided label smoothing
        output_real = netD(real_data)
```

```
lossD_real = criterion(output_real, label_real)
lossD_real.backward()

# Fake candidate generation
noise = torch.randn(b_size, NZ, 1, 1, device=device)
fake_data = netG(noise)
label_fake = torch.zeros((b_size,), dtype=torch.float, device=device)
output_fake = netD(fake_data.detach())
lossD_fake = criterion(output_fake, label_fake)
lossD_fake.backward()

lossD = lossD_real + lossD_fake
optimizerD.step()

# (2) Update Generator: maximize log(D(G(z)))
netG.zero_grad(set_to_none=True)
label_g = torch.ones((b_size,), device=device)
output_g = netD(fake_data)
lossG = criterion(output_g, label_g)
lossG.backward()
optimizerG.step()

epoch_d_loss += lossD.item()
epoch_g_loss += lossG.item()

avg_d = epoch_d_loss / N_BATCHES_PER_EPOCH
avg_g = epoch_g_loss / N_BATCHES_PER_EPOCH
d_losses.append(avg_d)
g_losses.append(avg_g)

# Checkpoint recording via genuine netG(fixed_noise) forward pass
if epoch in target_epochs.values():
    pct = [k for k, v in target_epochs.items() if v == epoch][0]
    netG.eval()
    with torch.no_grad():
        gen_out = netG(fixed_noise).detach().cpu()
        gen_imgs = (gen_out.permute(0, 2, 3, 1).numpy() * 0.5 + 0.5).clip(0, 1)
        saved_images_by_pct[pct] = [gen_imgs[i] for i in range(5)]
```

```

        netG.train()
        print(f"[{pct}% Progress | Epoch {epoch:3d}/{NUM_EPOCHS}] Loss_D: {avg_d:.4f}, Loss_G: {avg_g:.4f}", flush=True)
    elif epoch % 25 == 0:
        print(f"[Epoch {epoch:3d}/{NUM_EPOCHS}] Loss_D: {avg_d:.4f}, Loss_G: {avg_g:.4f}", flush=True)

# Cache checkpoints
cache_file = os.path.join(out_dir, "dcgan_faces_checkpoints.npz")
np.savez(cache_file,
         saved_images=np.array([saved_images_by_pct[p] for p in checkpoints_progress]),
         checkpoints_progress=np.array(checkpoints_progress),
         d_losses=np.array(d_losses),
         g_losses=np.array(g_losses))
print(f"Checkpoints cached to {cache_file}", flush=True)

# -----
# 5. Publication Visualization (5x11 Grid + Real Samples + Loss Dynamics)
# -----
print("Training complete! Generating publication figure...", flush=True)

plt.rcParams['font.sans-serif'] = 'Helvetica', 'DejaVu Sans', 'Arial'
plt.rcParams['mathtext.fontset'] = 'cm'

fig = plt.figure(figsize=(25.0, 14.8), dpi=300)
gs_main = gridspec.GridSpec(2, 1, height_ratios=[2.2, 0.95], hspace=0.28)

# Top Grid: 5 rows (Seeds) x 11 cols (Progress %)
gs_top = gridspec.GridSpecFromSubplotSpec(5, 11, subplot_spec=gs_main[0], hspace=0.08, wspace=0.08)

fig.suptitle(f"DCGAN Training Progression on Aligned Human Faces ({NUM_EPOCHS} Epochs, $64 \times 64$ Resolution)",
            fontsize=26, fontweight='bold', color='#0F172A', y=0.985)

for row in range(5):
    for col, pct in enumerate(checkpoints_progress):
        ax = fig.add_subplot(gs_top[row, col])
        img = saved_images_by_pct[pct][row]
        ax.imshow(img)
        ax.set_xticks([])
        ax.set_yticks([])

```

```

# Column Headers (Progress %) on Top Row
if row == 0:
    ep_text = f"Epoch {target_epochs[pct]}" if pct > 0 else "Untrained"
    ax.set_title(f"{pct}%\n({ep_text})", fontsize=15, fontweight='bold', color='#1E40AF', pad=8)

# Row Labels (Latent Vector Seed #) on Leftmost Column
if col == 0:
    ax.set_ylabel(f"Seed #{row+1}", fontsize=15, fontweight='bold', color='#1E3A8A', labelpad=8)

# Subtle border styling
for spine in ax.spines.values():
    spine.set_edgecolor('#CBD5E1')
    spine.set_linewidth(1.5)

# Bottom Section: Left = Real Dataset Samples, Right = Loss Dynamics Curves
gs_bottom = gridspec.GridSpecFromSubplotSpec(1, 2, subplot_spec=gs_main[1], width_ratios=[1.15, 1.05], wspace=0.25)

# Bottom Left: Real Training Samples Gallery (1 row x 8 real images at 64x64)
ax_real_container = fig.add_subplot(gs_bottom[0])
ax_real_container.axis('off')
ax_real_container.set_title("Authentic Training Dataset Samples (Ground Truth $64 \times 64$ Aligned Faces)",
                           fontsize=18, fontweight='bold', color='#166534', pad=12)

gs_real = gridspec.GridSpecFromSubplotSpec(1, 8, subplot_spec=gs_bottom[0], hspace=0.05, wspace=0.08)

# Select 8 diverse real face samples
sample_indices = [0, 800, 1600, 2400, 3200, 4000, 4800, 5600]
crop_sample = transforms.Compose([
    transforms.CenterCrop(140),
    transforms.Resize(64),
])

for i, s_idx in enumerate(sample_indices):
    ax_r = fig.add_subplot(gs_real[0, i])
    path = dataset.raw_paths[s_idx]
    real_img = crop_sample(Image.open(path).convert('RGB'))
    ax_r.imshow(real_img)

```

```

    ax_r.set_xticks([])
    ax_r.set_yticks([])
    ax_r.set_xlabel(f"Sample {i+1}", fontsize=12, fontweight='bold', color='#166534', labelpad=4)
    for spine in ax_r.spines.values():
        spine.set_edgecolor('#16A34A')
        spine.set_linewidth(2.0)

# Bottom Right: Generator vs. Discriminator Loss Curves on Dual Y-Axes (twinx)
ax_loss_D = fig.add_subplot(gs_bottom[1])
ax_loss_G = ax_loss_D.twinx()

epochs_x = np.arange(1, NUM_EPOCHS + 1)

line_D = ax_loss_D.plot(epochs_x, d_losses, label=r"Discriminator Loss  $\mathcal{L}_D$  (Left Axis)",
                        color='#EA580C', lw=3.0, zorder=3)
ax_loss_D.set_ylabel(r"Discriminator Loss  $\mathcal{L}_D$ ", fontsize=15, fontweight='bold', color='#EA580C')
ax_loss_D.tick_params(axis='y', labelcolor='#EA580C', labelsz=13)
ax_loss_D.tick_params(axis='x', labelsz=13)
ax_loss_D.set_xlabel("Training Epoch", fontsize=15, fontweight='bold', color='#334155')

line_G = ax_loss_G.plot(epochs_x, g_losses, label=r"Generator Loss  $\mathcal{L}_G$  (Right Axis)",
                        color='#1D4ED8', lw=3.0, zorder=3)
ax_loss_G.set_ylabel(r"Generator Loss  $\mathcal{L}_G$ ", fontsize=15, fontweight='bold', color='#1D4ED8')
ax_loss_G.tick_params(axis='y', labelcolor='#1D4ED8', labelsz=13)

# Mark 10% checkpoints on the curve
for pct in [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]:
    ep = target_epochs[pct]
    ax_loss_D.axvline(x=ep, color='#94A3B8', linestyle=':', alpha=0.5, lw=1.5)

ax_loss_D.set_title("Adversarial Minimax Loss Dynamics (Dual  $y$ -Axes)", fontsize=18, fontweight='bold', color='#0F172A', pad=10)
ax_loss_D.grid(True, linestyle='--', alpha=0.35)

# Combined Legend
lines = line_D + line_G
labels = [l.get_label() for l in lines]
ax_loss_D.legend(lines, labels, loc='upper right', fontsize=13, framealpha=0.95)

```

```
out_file = os.path.join(out_dir, "fig_6_03f_dcgan_faces_progression.png")
plt.savefig(out_file, dpi=300, bbox_inches='tight', pad_inches=0.15)
plt.close()
print(f"Saved: {out_file}", flush=True)
```

1.3.4.3 Execution Logs: Training Progression & Loss Dynamics

```
Using compute device: mps
Loading Aligned Human Faces dataset into memory...
Dataset ready: torch.Size([26466, 3, 64, 64]) (13,233 original + 13,233 flipped)
Generator total parameters: 3,576,704
Discriminator total parameters: 2,765,568
Starting DCGAN training on ALL 26,466 images for 1000 epochs on mps...
[Epoch 25/1000] Loss_D: 0.8328, Loss_G: 4.5049
[Epoch 50/1000] Loss_D: 0.8332, Loss_G: 3.6637
[Epoch 75/1000] Loss_D: 0.8231, Loss_G: 3.1868
[10% Progress | Epoch 100/1000] Loss_D: 0.7841, Loss_G: 2.7725
[Epoch 125/1000] Loss_D: 0.8061, Loss_G: 2.8720
[Epoch 150/1000] Loss_D: 0.7582, Loss_G: 2.6869
[Epoch 175/1000] Loss_D: 0.8037, Loss_G: 2.7212
[20% Progress | Epoch 200/1000] Loss_D: 0.7379, Loss_G: 2.7605
[Epoch 225/1000] Loss_D: 0.6898, Loss_G: 2.7233
[Epoch 250/1000] Loss_D: 0.7474, Loss_G: 2.7437
[Epoch 275/1000] Loss_D: 0.6215, Loss_G: 2.7862
[30% Progress | Epoch 300/1000] Loss_D: 0.6676, Loss_G: 2.8844
[Epoch 325/1000] Loss_D: 0.6566, Loss_G: 3.0102
[Epoch 350/1000] Loss_D: 0.5671, Loss_G: 3.0639
[Epoch 375/1000] Loss_D: 0.6489, Loss_G: 3.0808
[40% Progress | Epoch 400/1000] Loss_D: 0.5702, Loss_G: 3.1236
[Epoch 425/1000] Loss_D: 0.6065, Loss_G: 3.2776
[Epoch 450/1000] Loss_D: 1.3132, Loss_G: 3.2637
[Epoch 475/1000] Loss_D: 0.6259, Loss_G: 3.4630
[50% Progress | Epoch 500/1000] Loss_D: 0.5757, Loss_G: 3.3846
[Epoch 525/1000] Loss_D: 0.5295, Loss_G: 3.3996
[Epoch 550/1000] Loss_D: 0.7613, Loss_G: 3.0834
[Epoch 575/1000] Loss_D: 0.6517, Loss_G: 3.3156
[60% Progress | Epoch 600/1000] Loss_D: 0.5489, Loss_G: 3.5058
```

```
[Epoch 625/1000] Loss_D: 0.4645, Loss_G: 3.6504
[Epoch 650/1000] Loss_D: 0.5109, Loss_G: 3.7445
[Epoch 675/1000] Loss_D: 0.6692, Loss_G: 3.7424
[70% Progress | Epoch 700/1000] Loss_D: 0.5749, Loss_G: 3.6854
[Epoch 725/1000] Loss_D: 0.5648, Loss_G: 3.4738
[Epoch 750/1000] Loss_D: 0.4972, Loss_G: 3.6065
[Epoch 775/1000] Loss_D: 0.5246, Loss_G: 3.7430
[80% Progress | Epoch 800/1000] Loss_D: 0.4708, Loss_G: 3.9367
[Epoch 825/1000] Loss_D: 0.5222, Loss_G: 3.8286
[Epoch 850/1000] Loss_D: 0.4711, Loss_G: 4.0244
[Epoch 875/1000] Loss_D: 0.5045, Loss_G: 3.8035
[90% Progress | Epoch 900/1000] Loss_D: 0.4657, Loss_G: 4.0960
[Epoch 925/1000] Loss_D: 0.4567, Loss_G: 3.9221
[Epoch 950/1000] Loss_D: 0.4677, Loss_G: 3.8309
[Epoch 975/1000] Loss_D: 0.5056, Loss_G: 4.0993
[100% Progress | Epoch 1000/1000] Loss_D: 0.4260, Loss_G: 4.0618
Checkpoints cached to section/ch06_figures/dcgan_faces_checkpoints.npz
Training complete! Generating publication figure...
Saved: section/ch06_figures/fig_6_03f_dcgan_faces_progression.png
```

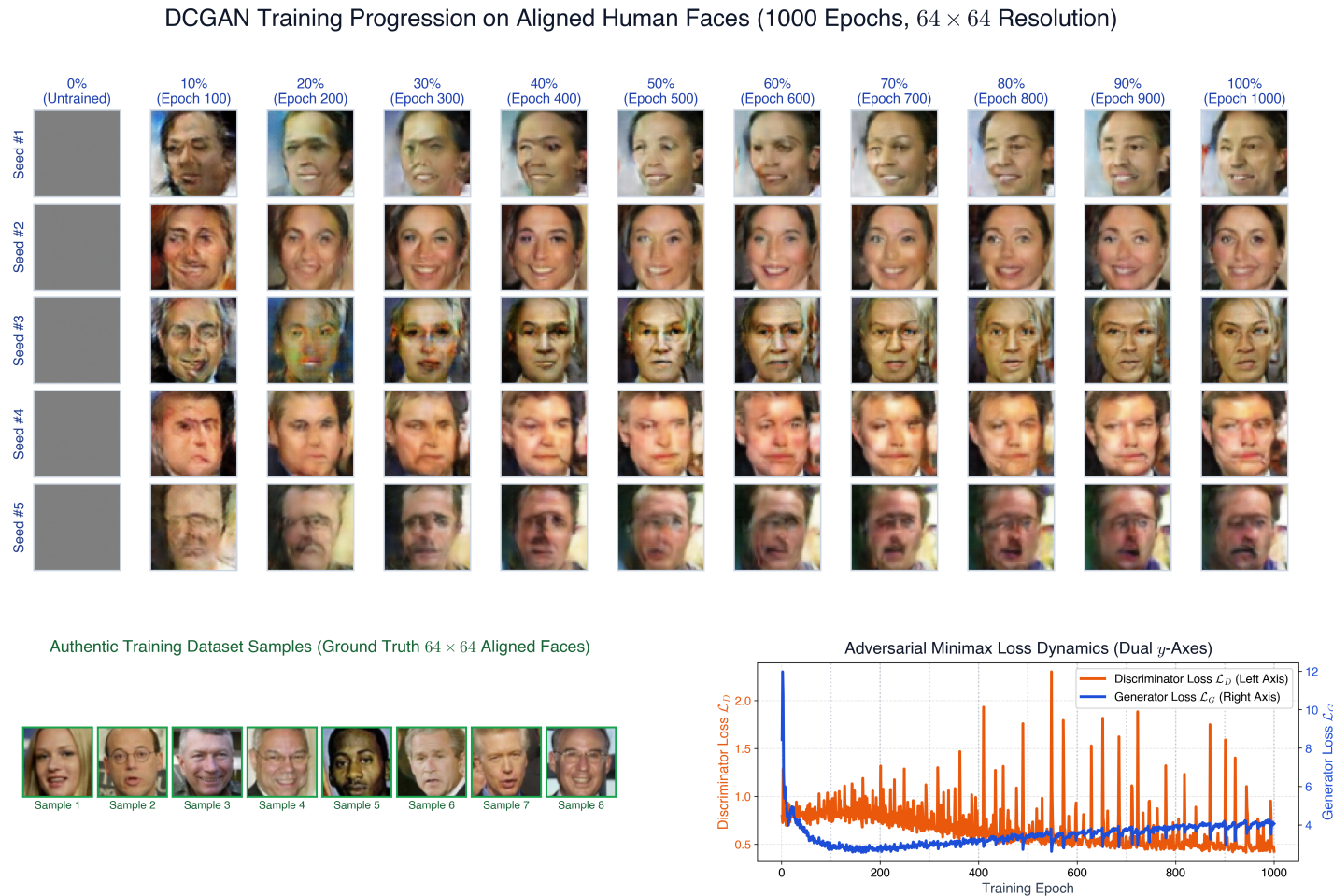


Figure 25: DCGAN Training Progression on Aligned Human Faces (1000 Epochs, 64×64 Resolution, 26,466 Augmented Samples): Tracking 5 fixed latent noise vectors ($\mathbf{z}_1, \dots, \mathbf{z}_5 \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{100})$) across training progress at every 10% interval (from 0% untrained random noise up to 100% completed training at Epoch 1000 via direct `netG(fixed_noise)` forward passes). Lower panels illustrate authentic ground-truth training samples (facial landmarks centered) and the dual y -axes adversarial loss dynamics: **Discriminator Loss** ($\mathcal{L}_D$, left axis / orange) penalizes incorrect classification of real images ($D(\mathbf{x}) \rightarrow 1$) and generated fakes ($D(G(\mathbf{z})) \rightarrow 0$), while **Generator Loss** ($\mathcal{L}_G$, right axis / blue) measures how effectively synthesized faces fool the Discriminator into predicting real authenticity ($D(G(\mathbf{z})) \rightarrow 1$) via non-saturating cross-entropy.

1.3.5 Applications of GANs

GANs can be used to generate realistic data and transform existing data into new forms.

1.3.5.1 Image Generation

GANs can generate realistic images, such as:

- Human faces and characters.
- High-resolution images with realistic details.

1.3.5.2 Synthetic Data Generation

GANs can generate additional training data when real data are limited, such as:

- Medical images for rare diseases.
- Driving scenes with unusual weather or road conditions.

1.3.5.3 Image-to-Image Translation

GANs can transform images from one style or domain to another, such as:

- Sketches into realistic images.
- Daytime images into nighttime images.
- Photos into artistic styles.

1.3.5.4 Other Applications

GAN-based adversarial methods can also be used to test AI systems by generating difficult or unusual inputs, helping improve their robustness and safety.

GAN Design

- **Sharp Details:** Because the Discriminator penalizes unnatural blurring, GANs produce significantly sharper images than standard Autoencoders.
- **Learned Loss Function:** Instead of relying on a fixed mathematical formula (like MSE), the loss function is learned adaptively by the Discriminator.

Implementation difficulties (Cheating in learning): Mode Collapse & Training Instability

- **Mode Collapse:** A failure mode where the Generator finds a small number of specific samples that reliably fool the Discriminator and repeatedly outputs only those samples, ignoring the diversity of the true dataset.
- **Training Instability:** If the Discriminator learns too quickly, its classification gradients vanish, leaving the Generator with no useful learning signal.

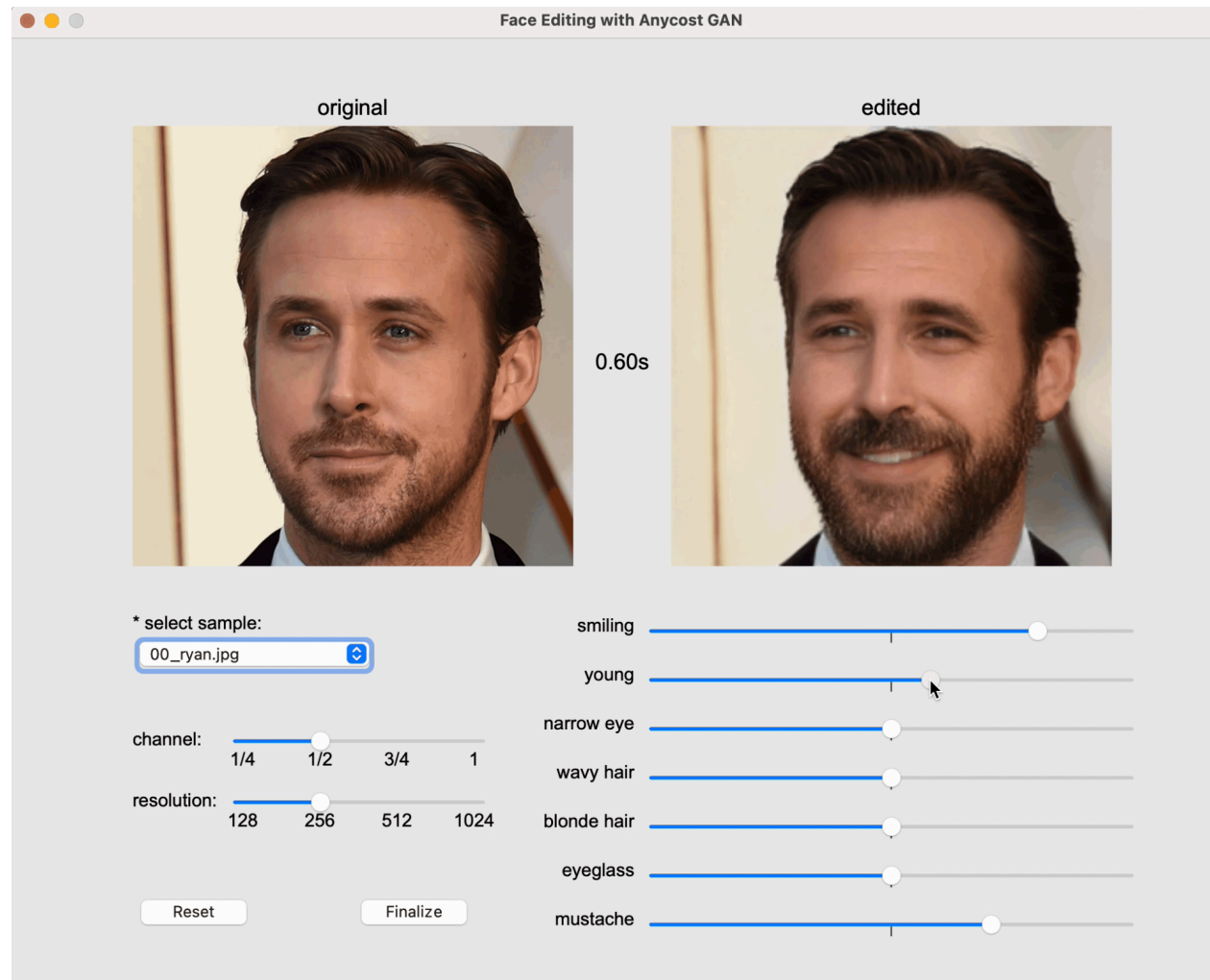


Figure 26: Interactive Facial Attribute Editing via Generative Adversarial Networks: Continuous latent space traversal enables real-time attribute manipulation (e.g., age, smiling, facial hair) on high-resolution portrait imagery using Anycost GAN. (Source: <https://developer.ibm.com/articles/generative-adversarial-networks-explained/>)

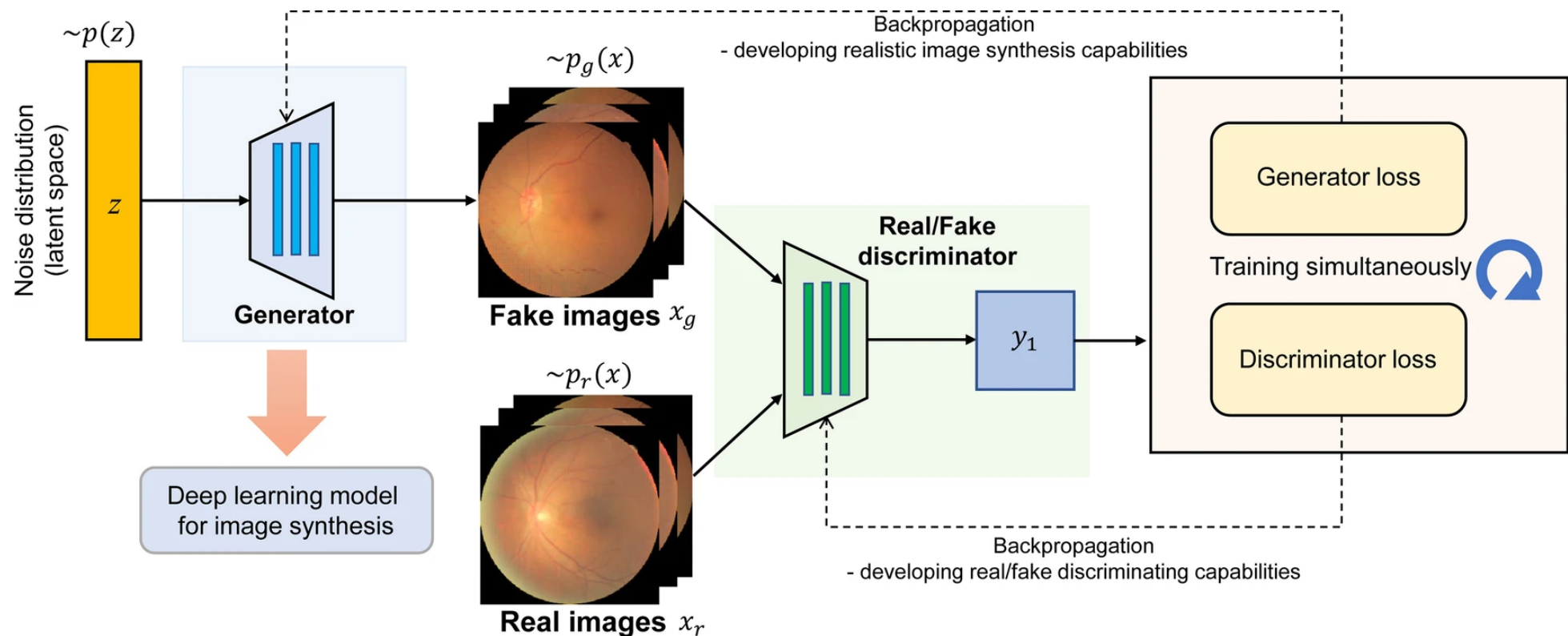


Figure 27: Medical Imaging Applications of Generative Adversarial Networks: Synthesis of pathological retinal fundus photographs from latent noise $z \sim p(z)$ to augment training data for ophthalmic diagnostic systems, where the Generator and Real/Fake Discriminator optimize simultaneously through joint loss backpropagation. (Source: <https://link.springer.com/article/10.1186/s40662-022-00277-3#Fig1>)

1.4 Recurrent Neural Networks (Sequential Learning)

1.4.1 Architectural Design: Learning from Sequential Data

Standard neural networks usually process each input independently. However, **sequential data** such as text, speech, and time-series data contains information that depends on the **order of the inputs**.

Recurrent Neural Networks (RNNs) are neural networks designed to process sequential data. They process inputs step-by-step while maintaining information from previous time steps in a hidden state, allowing the network to use the context of earlier inputs when processing the current input.

Long Short-Term Memory (LSTM) and **Gated Recurrent Units (GRUs)** are gated variants of RNNs designed to better retain and control information over longer sequences.

1.4.2 Why Sequential Data Is Different

Sequential data has several important properties:

- **Variable Length:** Sequences can have different lengths, such as short or long sentences.
- **Order Matters:** Changing the order of elements can change the meaning, such as “Dog bites man” and “Man bites dog.”
- **Past Information Matters:** Understanding the current input may require information from earlier in the sequence.

1.4.3 The Problem with Basic RNNs

A basic RNN processes a sequence step-by-step. At each time step, it combines the current input with information from the previous time step. This allows the network to use past information when processing the current input.

However, during training, the error signal is sent backward through many time steps. For long sequences, this signal can become very small as it is repeatedly propagated through the network. This is known as the **vanishing gradient problem**.

As a result, a basic RNN may have difficulty remembering important information from far earlier in a sequence.

In short:

A basic RNN can remember recent information, but it may gradually lose information from much earlier in a long sequence.

1.4.4 How LSTMs Learn

1.4.4.1 The Main Idea (Gated Memory): An LSTM uses a separate **cell state** together with **gates** to control the information stored in its memory.

- **Cell State (c_t):** Carries information across time steps.
- **Hidden State (h_t):** Represents the information used at the current time step.
- **Gates:** Control which information should be forgotten, added, or used.

1.4.4.2 The Three Gates: LSTM uses three main gates to control its memory:

1. **Forget Gate:** Decides which information from the previous memory should be removed.
2. **Input Gate:** Decides which new information should be added to the memory.
3. **Output Gate:** Decides which information from the memory should be used as the current output.

The gates use the sigmoid function to produce values between 0 and 1. A value close to 0 means that the information is mostly blocked, while a value close to 1 means that the information is mostly allowed through.

In short:

An LSTM reads a sequence step-by-step and learns what information to **remember, forget, and use**. This helps it handle information that needs to be remembered for longer periods.

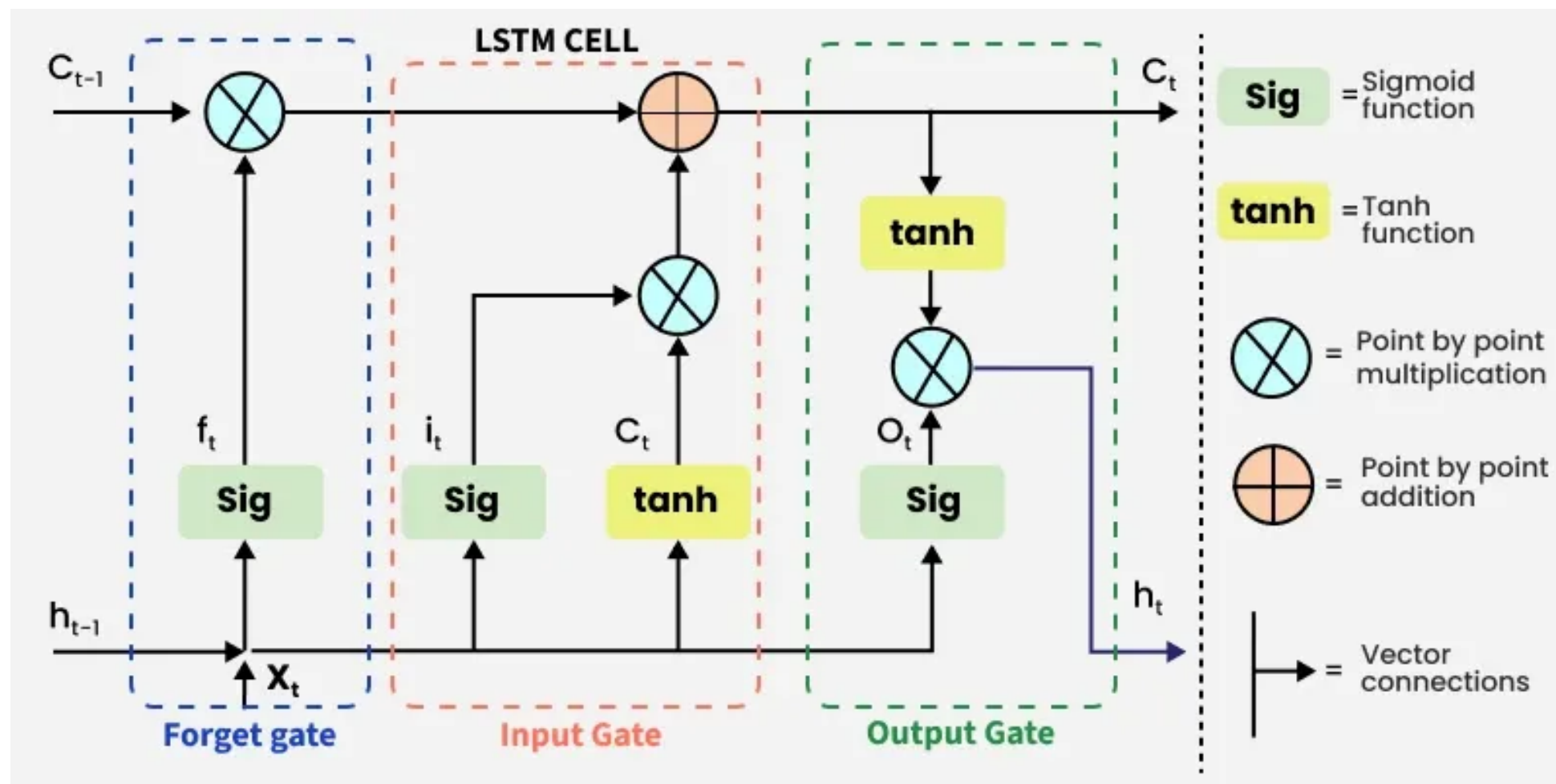


Figure 28: Internal Gating Architecture of an LSTM Cell: Detailed structural breakdown of the Forget Gate (f_t), Input Gate (i_t and candidate $\tilde{C}_t$), and Output Gate (o_t). Pointwise multiplication and addition operations regulate the flow through the cell state conveyor belt C_t to produce the final hidden state output h_t . (Source: <https://www.geeksforgeeks.org/deep-learning/deep-learning-introduction-to-long-short-term-memory/>)

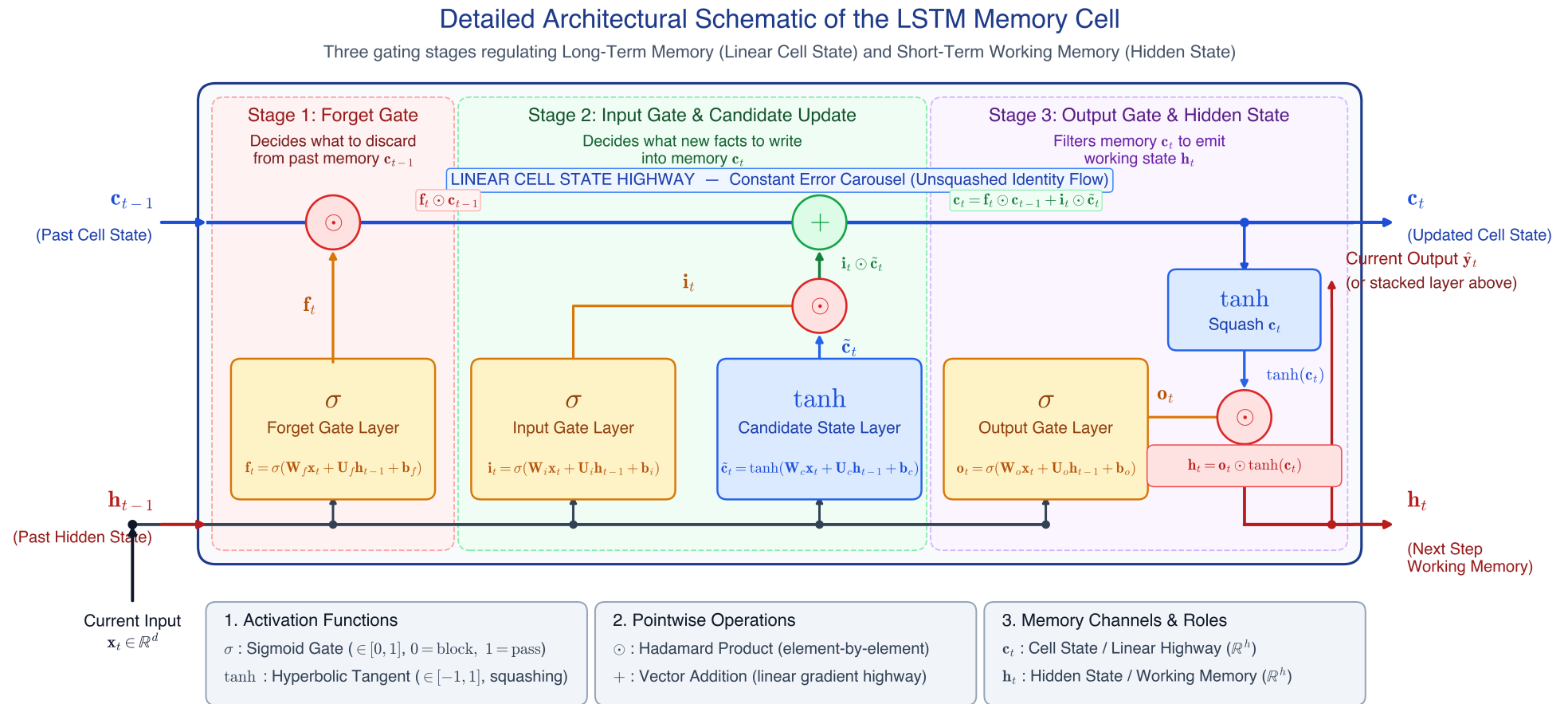


Figure 29: Detailed architectural schematic of the LSTM Memory Cell: linear Cell State (c_t) conveyor belt regulated by the Forget Gate (f_t), Input Gate (i_t), and Output Gate (o_t).

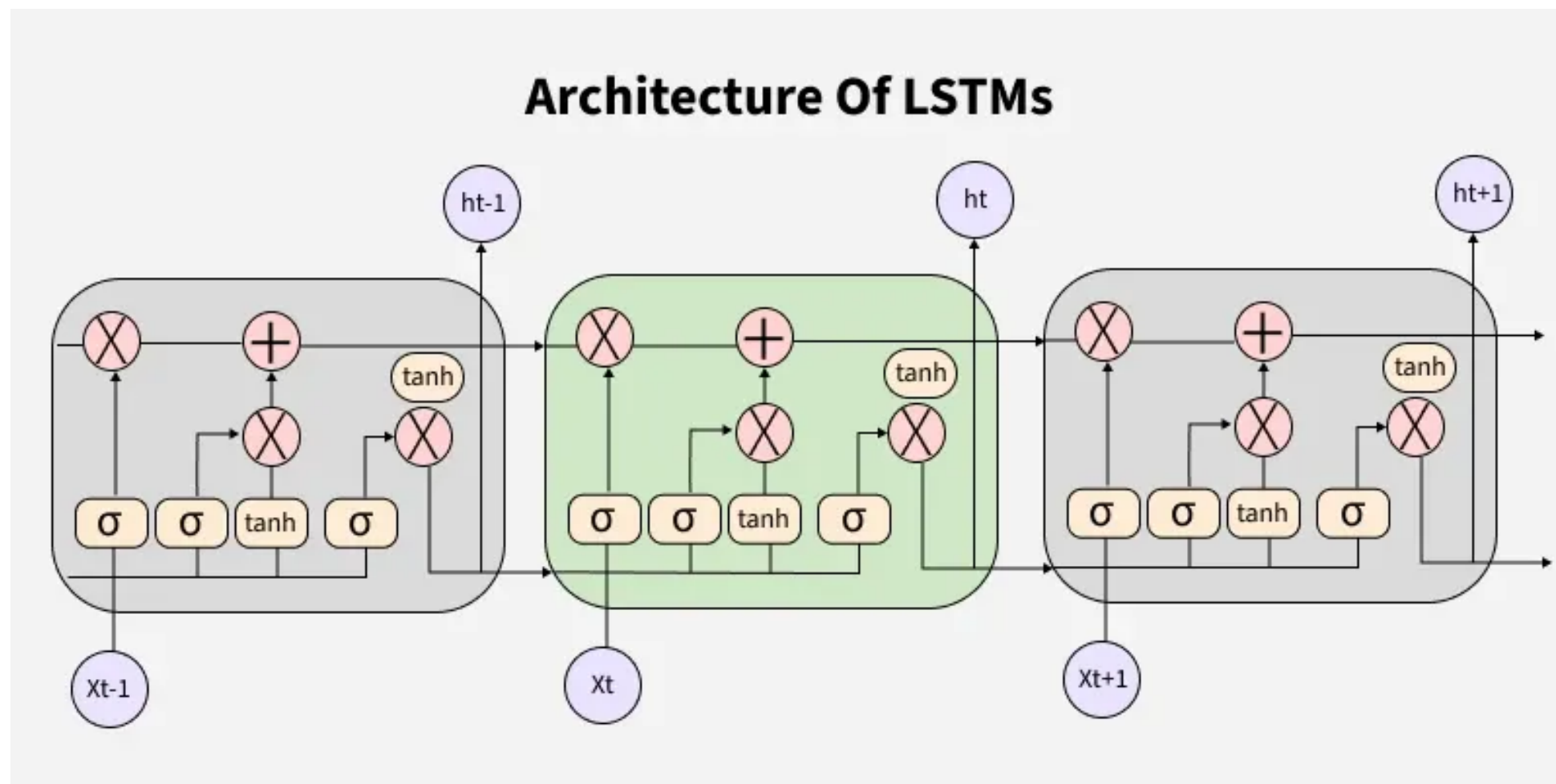


Figure 30: Architecture of Long Short-Term Memory (LSTM) Networks: Unrolled recurrent sequence processing across consecutive time steps $t - 1$, t , and $t + 1$. The horizontal line across the top carries the cell state memory, while the forget gate (σ), input gate (σ and $\tanh$), and output gate (σ and $\tanh$) interact to regulate information retention, candidate updates, and hidden state emission $\mathbf{h}_t$. (Source: <https://www.geeksforgeeks.org/deep-learning/deep-learning-introduction-to-long-short-term-memory/>)

1.4.5 Applications of RNNs and LSTMs

RNNs and LSTMs are designed for tasks involving **sequential data**, where the order of the inputs is important.

- **Natural Language Processing (NLP):** Processing text, language modeling, and text prediction.
- **Speech Processing:** Processing speech and audio sequences.
- **Time-Series Prediction:** Predicting values from sequences such as weather, sensor readings, and electricity demand.

For example, an RNN or LSTM can read a sentence word-by-word and use information from previous words to help predict the next word. This idea was an important foundation for early neural language models and sequence-to-sequence systems.

1.4.5.1 Sequence-to-Sequence Learning

RNNs and LSTMs can also be used in **Sequence-to-Sequence (Seq2Seq)** models, where one sequence is converted into another sequence.

- **Encoder:** Reads the input sequence and learns its representation.
- **Decoder:** Uses this representation to generate the output sequence.

For example, a Seq2Seq model can read an English sentence and generate its translation in Thai or another language.

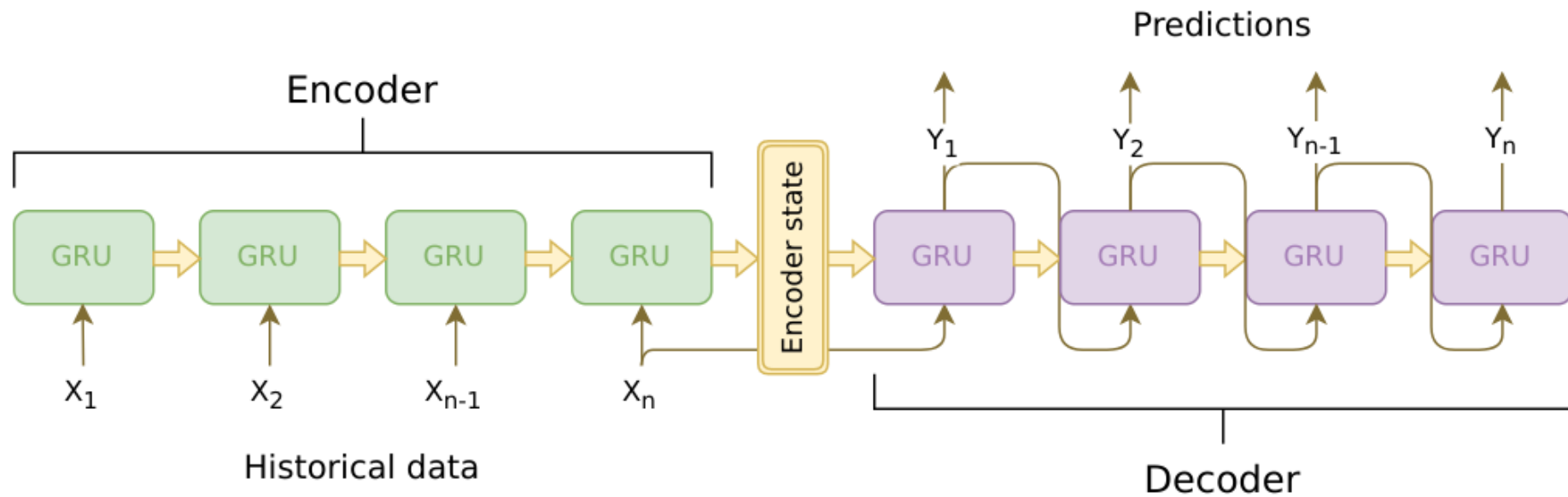


Figure 31: Sequence-to-Sequence (Seq2Seq) Architecture with Recurrent Encoder-Decoder Structure for Time-Series and Sequential Prediction (Source: <https://github.com/sooftware/seq2seq>).

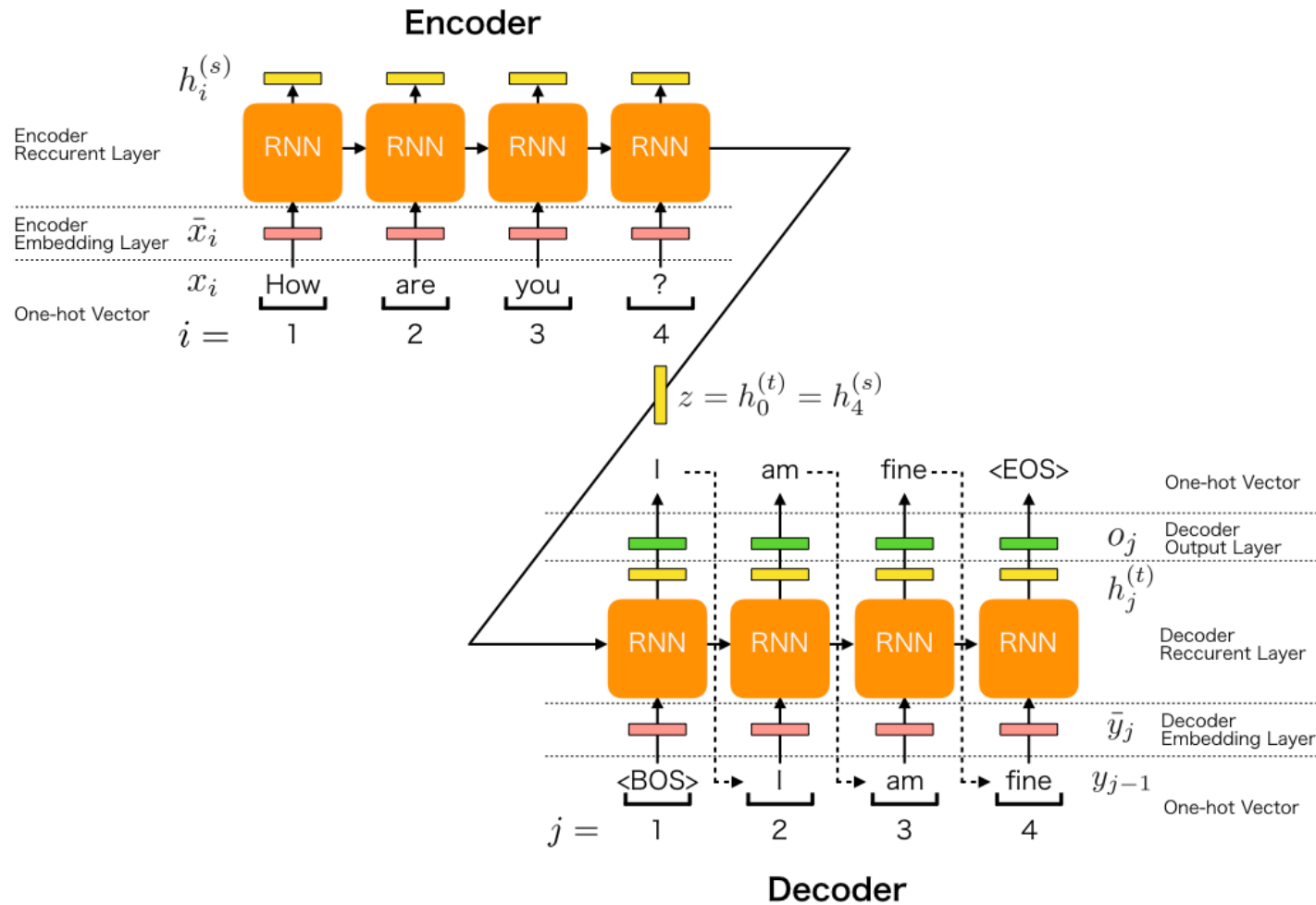


Figure 32: Architecture of a Sequence-to-Sequence (Seq2Seq) Model: The recurrent Encoder reads the variable-length input sequence and compresses it into a fixed-dimensional context vector $z = h_4^{(s)}$, which subsequently initializes the hidden state of the recurrent Decoder ($h_0^{(t)} = z$) to generate target sequence tokens autoregressively (Source: <https://docs.chainer.org/en/v7.8.0/examples/seq2seq.html>).

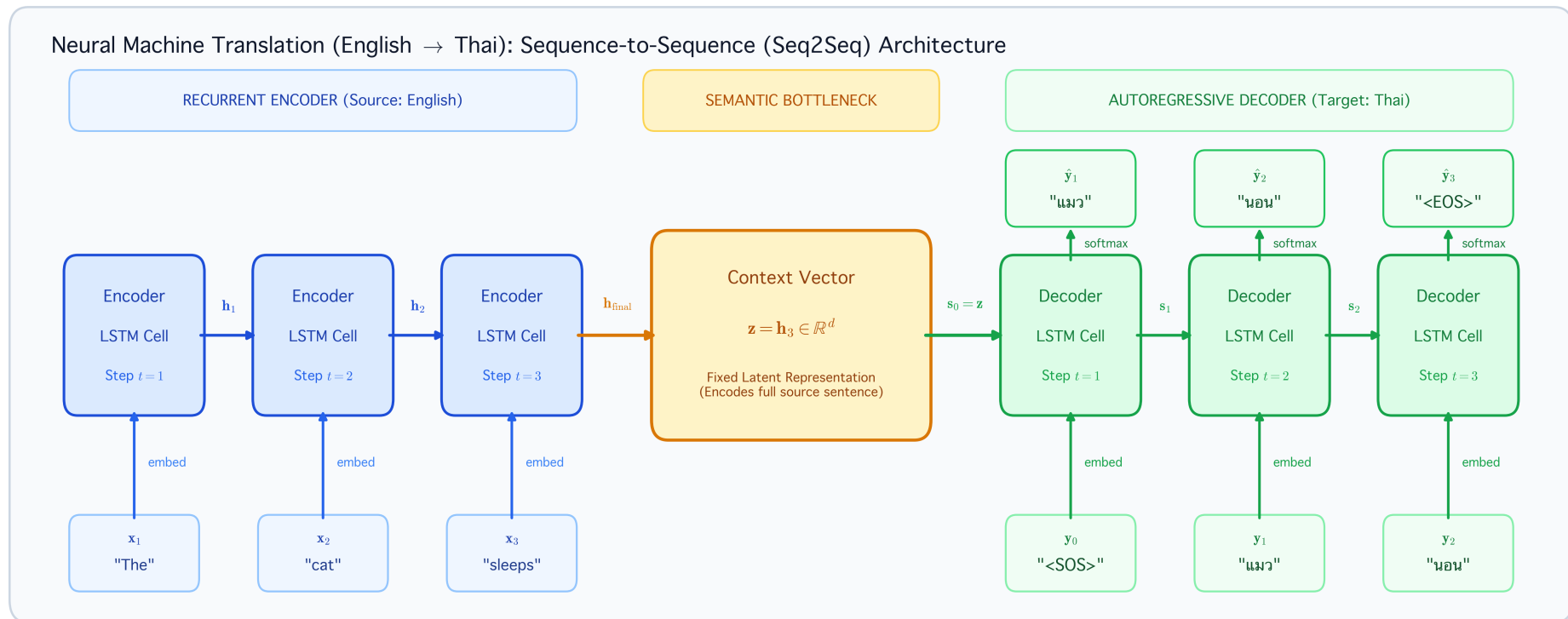


Figure 33: Sequence-to-Sequence (Seq2Seq) Neural Machine Translation Pipeline (English → Thai): (1) **Recurrent Encoder:** The English source tokens (x_1 = “The”, x_2 = “cat”, x_3 = “sleeps”) are processed step-by-step through recurrent LSTM cells, recursively updating hidden states $h_t = f(h_{t-1}, x_t)$ to capture context across time; (2) **Semantic Bottleneck:** The final encoder hidden state $h_{\text{final}} = h_3$ serves as a dense context vector $z \in \mathbb{R}^d$ that encapsulates the full semantic meaning and syntax of the entire source sentence; (3) **Autoregressive Decoder:** Initialized with the context vector ($s_0 = z$), the recurrent Decoder emits target Thai tokens ($\hat{y}_1$ = “แมว”, $\hat{y}_2$ = “นอน”, $\hat{y}_3$ = “<EOS>”) sequentially via Softmax probability distributions, where each generated token (or teacher-forcing input y_{t-1}) is fed autoregressively into the next recurrent step until the End-of-Sequence token (<EOS>) is produced.

1.4.5.2 Language Modeling and Text Prediction

In language modeling, the model predicts the next word or token based on the previous sequence.

$$P(w_t \mid w_1, w_2, \dots, w_{t-1}) \quad (7)$$

RNNs and LSTMs can perform this task by processing the text sequentially and carrying information from previous words. This idea was later extended by **Attention** and **Transformer** architectures, which became the foundation of modern Large Language Models (LLMs).

1. **RNNs and LSTMs:** Process sequences step-by-step and use previous information to understand the current input. However, their sequential processing creates limitations for handling very long sequences and training large models.
2. **Attention:** Allows a model to directly focus on relevant information from different parts of a sequence.
3. **Transformers:** Use self-attention to process many tokens in parallel during training. This makes them highly effective for large-scale language models.

Modern language models are based primarily on **Transformer architectures** rather than traditional RNNs or LSTMs:

- **BERT (Bidirectional Encoder):** Analyzes text in both directions simultaneously to learn deep contextual representations, widely used for text classification, semantic search, and embeddings.
- **Modern Generative LLMs (GPT, Gemini, Claude):** Use autoregressive decoders to generate text step-by-step through next-token prediction.

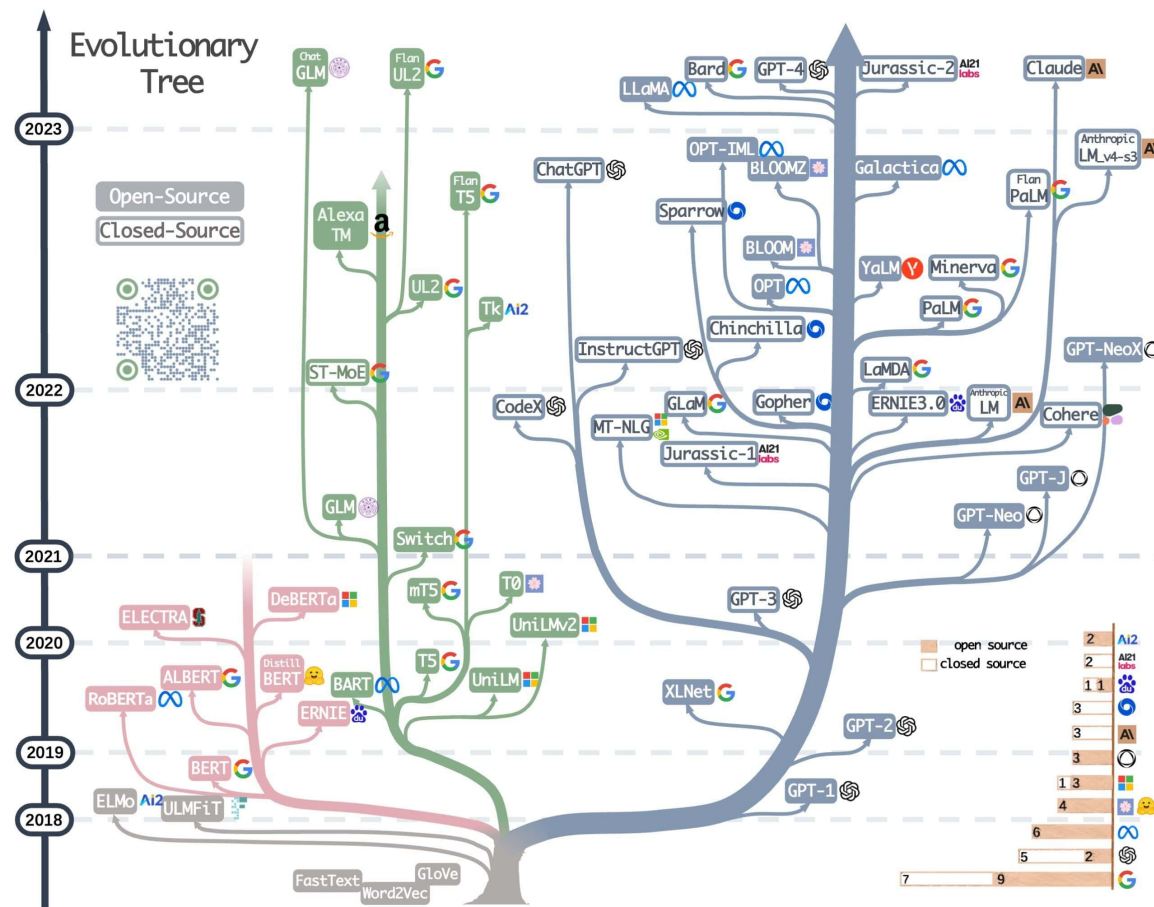


Figure 34: Evolutionary Tree of Modern Large Language Models (LLMs): Tracing the architectural progression from foundational word representations (Word2Vec, GloVe, ELMo) into three core Transformer paradigms: (1) Encoder-only (pink; e.g., BERT, RoBERTa) for bidirectional contextual representation; (2) Encoder-Decoder (green; e.g., T5, BART) for sequence transduction; and (3) Decoder-only (blue; e.g., GPT-1 through GPT-4, LLaMA, Claude) for autoregressive next-token prediction (Source: <https://www.baeldung.com/cs/bert-vs-gpt-3-architecture>).

1.4.5.3 Time-Series and Sensor Data

RNNs and LSTMs can also be applied to continuous sequential data, where the order of observations is important:

- **Wearable Sensor & Motion Tracking:** Continuous IMU accelerometry and gyroscopic data for human activity recognition and biomechanical analysis.
- **Weather Forecasting:** Using previous weather observations to predict future conditions.
- **Sensor Data:** Analyzing sequences of measurements from IoT devices and other sensors.
- **Energy Prediction:** Predicting electricity demand from historical usage and other measurements.
- **Biomedical Signals:** Analyzing signals such as ECG and other physiological measurements.

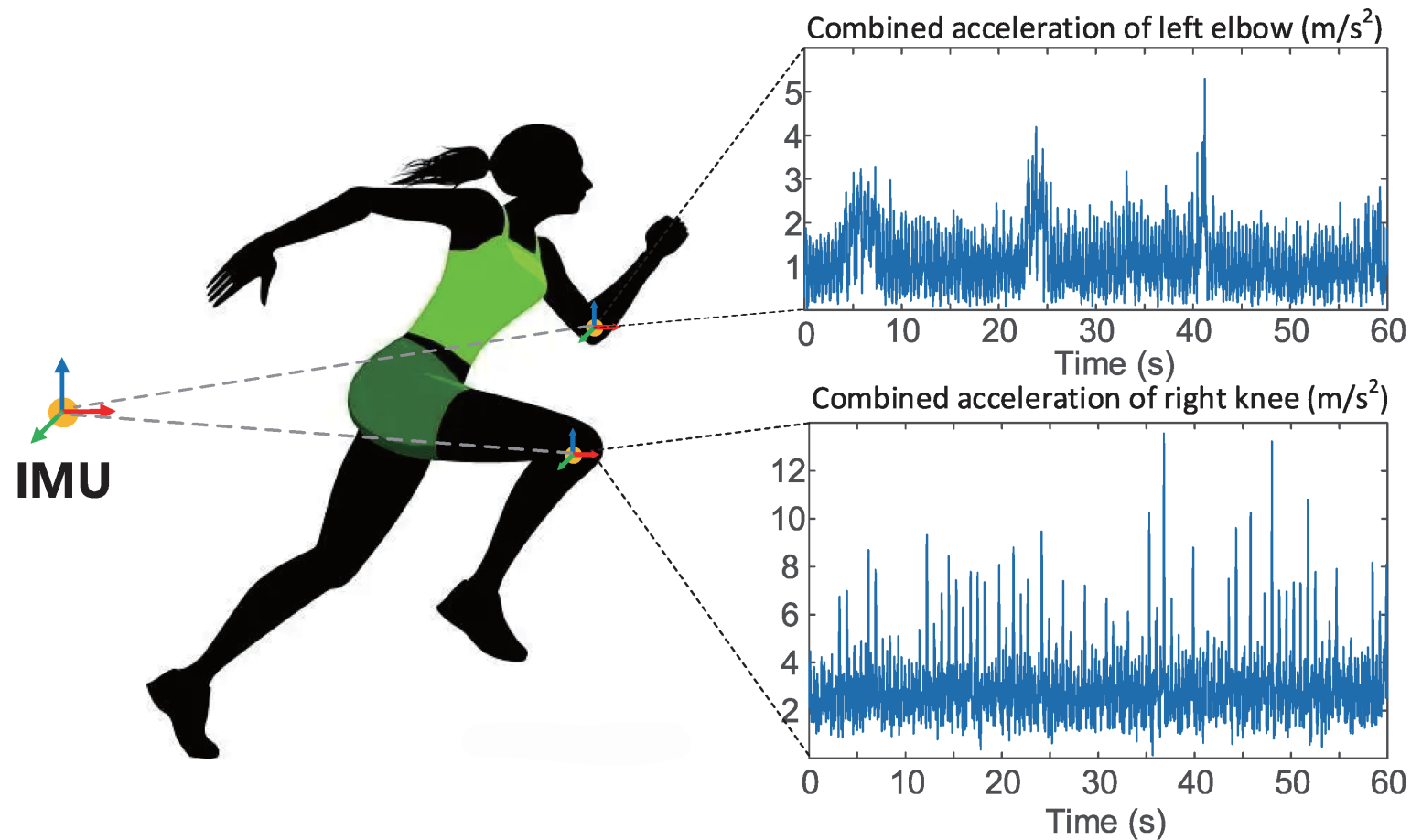


Figure 35: Wearable IMU Time-Series Sensor Stream: Inertial Measurement Units (IMUs) capturing multi-axis kinematic acceleration sequences (e.g., left elbow and right knee movement over time). Recurrent neural networks (RNNs/LSTMs) process these continuous multi-variate temporal waveforms to perform human activity recognition, gait analysis, and domain adaptation (Source: <https://www.mdpi.com/1424-8220/22/15/5507>).

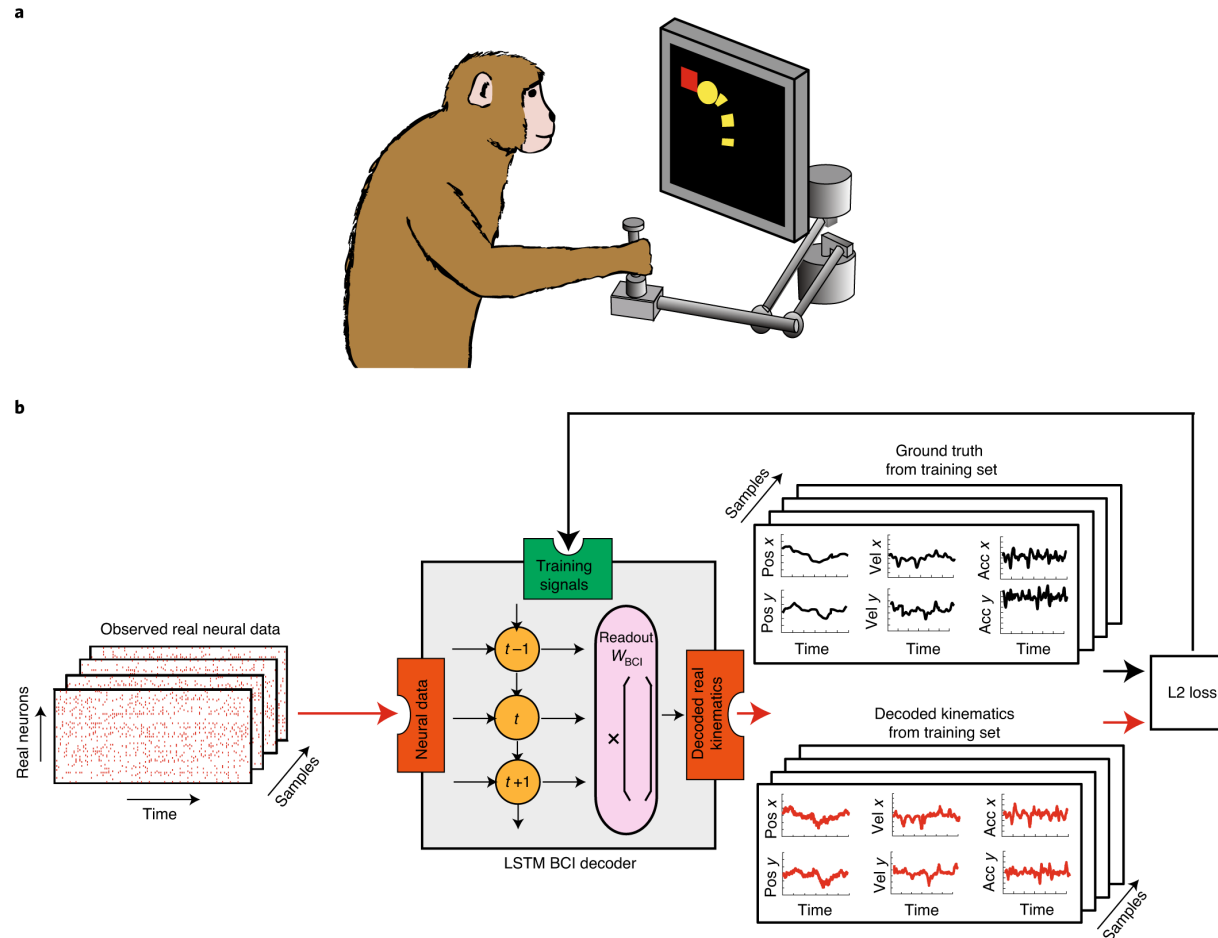


Figure 36: Biomedical Biosignal Decoding via Recurrent Neural Networks: (a) Experimental Brain-Computer Interface (BCI) motor reaching task; (b) Recurrent neural decoder architecture where multi-channel continuous neural firing rate time-series are processed sequentially across time steps $(t-1, t, t+1)$ by LSTM layers to decode real-time continuous movement kinematics (position, velocity, acceleration), optimized via L_2 regression loss against ground-truth trajectories (Source: <https://www.nature.com/articles/s41551-021-00811-z>).

1.5 Architectural Comparison: Autoencoders vs. GANs vs. LSTMs

Feature	Autoencoders (AE)	GANs	LSTMs
Main Purpose	Learn useful representations and reconstruct data	Generate realistic synthetic data	Model sequential data and long-term dependencies
Basic Structure	Encoder $\rightarrow$ Latent Space $\rightarrow$ Decoder	Generator $\leftrightarrow$ Discriminator	Recurrent cells with memory and gates
How It Learns	Minimize reconstruction error	Generator and Discriminator compete with each other	Learn from sequential input to predict the next output
Main Strength	Learns compact latent representations	Produces realistic generated samples	Handles long-term dependencies
Main Weakness	Reconstructions can be blurry	Training can be unstable	Training is difficult to parallelize
Typical Applications	Compression, denoising, anomaly detection	Image generation, data augmentation	Time series, speech, sequence modeling

Table 1: Simplified comparison of Autoencoders, GANs, and LSTMs.

Note: The five architectures mentioned up to now (**MLPs, CNNs, Autoencoders (AEs), GANs, and LSTMs**) are not mutually exclusive. Modern AI systems often combine these architectures for different tasks.

1.6 Practice Questions & Solutions

Practice Questions (1/2)

1. **Autoencoders in LLMs & RAG:** How is the concept of an Autoencoder bottleneck used in modern Retrieval-Augmented Generation (RAG) and vector databases for Large Language Models?
2. **Industrial Anomaly Detection:** How can an Autoencoder detect defective microchips on an assembly line without ever being trained on defect examples?
3. **GAN Competitive Dynamics in LLMs:** How can the adversarial Generator vs. Discriminator game be applied to automate LLM Safety and Red-Teaming?
4. **Mode Collapse:** What is Mode Collapse in GANs, and why does it happen?

Practice Questions (2/2)

5. **Vanishing Gradients in RNNs:** Why do standard vanilla RNNs fail when processing long sequences of text, and how does the LSTM Cell State solve this?
6. **Language Modeling & LLMs:** How does next-token prediction work, and how does the sequential prediction concept in RNNs/LSTMs connect to modern Large Language Models (LLMs)?
7. **LSTM Gating Mechanism:** What are the specific roles of the Forget Gate, Input Gate, and Output Gate in managing memory flow inside an LSTM cell?
8. **Model Selection:** For each scenario below, choose whether an Autoencoder, a GAN, or an LSTM is the most appropriate architecture:
 - (a) Generating synthetic photorealistic human faces for virtual avatars.
 - (b) Building an autoregressive text autocompletion engine for a mobile keyboard.
 - (c) Flagging anomalous fraudulent credit card transactions in real time.

Solutions: Practice Questions (1/2)

1. **Autoencoders in LLMs & RAG:** Autoencoder bottleneck principles are used to compress high-dimensional text passages into dense 768- or 1536-dimensional semantic vector embeddings. In RAG systems, these embeddings are indexed in vector databases (like Pinecone). When a user prompts an LLM, the system performs fast cosine similarity search on the latent embeddings to retrieve factual reference documents, grounding the LLM's answers.
2. **Industrial Anomaly Detection:** The Autoencoder is trained exclusively on normal, defect-free items. It learns the standard visual patterns of normal products. When a defective item (with a crack or scratch) is evaluated, the network cannot reconstruct the anomaly because it was never exposed to defects. This causes a large reconstruction error ($\|\mathbf{x} - \hat{\mathbf{x}}\|^2 > \tau$), immediately flagging the item as defective.
3. **GAN Competitive Dynamics in LLMs:** In LLM safety red-teaming, an adversarial “Attacker” model is trained to generate complex prompt injection and jailbreak attacks, while a “Defender” LLM and reward model learn to identify and refuse harmful instructions, creating an automated adversarial hardening loop.
4. **Mode Collapse:** Mode Collapse occurs when the Generator discovers a small handful of specific images that reliably fool the Discriminator, and continuously generates only those few images while ignoring the full variety of the real dataset.

Solutions: Practice Questions (2/2)

5. **Vanishing Gradients in RNNs:** In vanilla RNNs, backpropagation through time repeatedly multiplies error gradients by the recurrent weight matrix, causing them to shrink exponentially toward zero over long sequences. LSTMs solve this by introducing an additive linear Cell State highway ($\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$), allowing gradients to travel back across hundreds of steps without decaying.
6. **Language Modeling & LLMs:** Language modeling trains a model to predict the probability distribution of the next token given all preceding tokens: $P(w_t \mid w_1, w_2, \dots, w_{t-1})$. RNNs and LSTMs established this foundation by updating a hidden state sequentially step-by-step. Modern Large Language Models (such as GPT, Gemini, and Claude) use the exact same next-token prediction objective, but employ Transformer self-attention to process and train on billions of tokens in parallel.
7. **LSTM Gating Mechanism:**
 - **Forget Gate ($\mathbf{f}_t$):** Decides what outdated or irrelevant past information to discard from the Cell State.
 - **Input Gate ($\mathbf{i}_t$):** Decides what new candidate information from the current input should be stored into the Cell State.
 - **Output Gate ($\mathbf{o}_t$):** Controls which parts of the updated Cell State are emitted as the hidden state output ($\mathbf{h}_t$) for the current step.
8. **Model Selection:**
 - (a) **GAN:** Best for synthesizing new, photorealistic image samples.
 - (b) **LSTM:** Best for sequential next-word autocompletion and time-series modeling.
 - (c) **Autoencoder:** Best for unsupervised anomaly detection via reconstruction error thresholding.

1.7 Chapter Summary

This chapter explored three advanced deep learning architectures designed for unsupervised representation learning, generative data synthesis, and sequential modeling:

- **1. Autoencoders (Data Compression & Representation Learning):**
 - **Structure:** Consists of an **Encoder** ($f_{\theta}(\mathbf{x})$) that maps input data into a compact bottleneck **Latent Space** ($\mathbf{z}$), and a **Decoder** ($g_{\phi}(\mathbf{z})$) that reconstructs the original data ($\hat{\mathbf{x}}$).
 - **Learning Objective:** Minimizes unsupervised reconstruction error ($\|\mathbf{x} - \hat{\mathbf{x}}\|^2$), forcing the network to discover the most informative low-dimensional representations without manual labels.
 - **Key Applications:** Image denoising (Denoising Autoencoders / DAE), industrial anomaly detection via reconstruction error thresholding ($\|\mathbf{x} - \hat{\mathbf{x}}\|^2 > \tau$), and dense vector embeddings for vector search and Retrieval-Augmented Generation (RAG).
- **2. Generative Adversarial Networks (Adversarial Data Generation):**
 - **Structure:** A two-player game pairing a **Generator** (G) that synthesizes fake samples from random noise against a **Discriminator** (D) that classifies samples as real or fake.
 - **Learning Objective:** Trained through minimax adversarial optimization ($\min_G \max_D V(D, G)$), where the Discriminator provides a learned, adaptive loss function that guides the Generator toward photorealistic outputs.
 - **Key Applications & Challenges:** High-resolution image synthesis, synthetic data augmentation, and image-to-image translation. Key implementation challenges include training instability and mode collapse.

- **3. LSTMs & Recurrent Sequence Models (Sequential Learning & Memory):**

- **Structure & Gating:** Overcomes the vanishing gradient limitation of basic vanilla RNNs using a dual-state architecture: a long-term **Cell State** ($\mathbf{c}_t$) and a working **Hidden State** ($\mathbf{h}_t$), regulated by three learnable sigmoid gates:
 - * **Forget Gate** ($\mathbf{f}_t$): Discards outdated information from the Cell State.
 - * **Input Gate** ($\mathbf{i}_t$): Stores approved candidate information into the Cell State.
 - * **Output Gate** ($\mathbf{o}_t$): Determines the emitted hidden state output.
- **Key Applications & LLM Connection:** Sequence-to-Sequence (Seq2Seq) encoder-decoder models, next-token prediction language modeling ($P(w_t \mid w_{<t})$) providing the conceptual foundation for modern Large Language Models (LLMs), and continuous time-series/sensor tracking (e.g., wearable IMUs and BCI neural decoding).

- **4. Architectural Complementarity & Hybridization:**

- The foundational architectures covered up to now (MLPs, CNNs, Autoencoders, GANs, LSTMs, and modern Transformers) are not mutually exclusive. Modern AI systems frequently combine them (e.g., VAE-GANs, TimeGAN, and CNN-LSTM/Transformer pipelines) to solve complex multimodal tasks.

Together, these architectures expand deep learning beyond static classification into representation compression, generative modeling, and temporal sequence reasoning.